INTRODUCTION TO ANDROID / SIS3

RateRadar KZ

От первой строки до самостоятельной защиты

Kotlin с нуля. Android и Compose.

Каждый файл и каждая конструкция.

Путь нажатия, состояния и навигация.

Аудит задания и практика изменений.

```
MainActivity
  -> RateRadarTheme
     -> RateRadarNavGraph
        -> Screens -> Components
```

Учебное пособие на основе твоего реального проекта

Проверка и разбор: 9 октября 2026 года, Алматы

Сначала понимание, затем изменения без подсказки.

Оглавление

Как пользоваться этим разбором 4

Что именно уже делает приложение 5

Kotlin с нуля: как читать эти файлы 7

Android: что происходит от Run до экрана 11

Compose: как из функций получается интерфейс 13

Состояние: почему экран меняется после нажатия 16

Файловая иерархия: что за что отвечает 18

Данные и точка входа: разбор файлов 19

- ExchangeOffice.kt . . . 19
- SampleData.kt . . . 21
- MainActivity.kt . . . 23

Тема и размеры: как менять оформление в одном месте 25

- Dimens.kt . . . 25
- Color.kt . . . 27
- Theme.kt . . . 31
- Type.kt . . . 33

Переиспользуемые компоненты: разбираем внутренности 34

- RateBlock.kt . . . 34
- OfficeCard.kt . . . 37
- EmptyState.kt . . . 42

Три экрана: разбор каждого блока 45

- ExchangeOfficeListScreen.kt . . . 45
- FavoritesScreen.kt . . . 50
- OfficeDetailsScreen.kt . . . 54

Навигация: как id доходит до деталей 61

- RateRadarNavGraph.kt . . . 61

Три сценария: проследи данные руками 66

Ресурсы, Manifest и Gradle: разбор служебных файлов 67

AndroidManifest.xml . . . 67
settings.gradle.kts . . . 68
build.gradle.kts . . . 69
build.gradle.kts . . . 69
gradle.properties . . . 73
gradle-wrapper.properties . . . 73
ExampleUnitTest.kt . . . 75
strings.xml . . . 75
themes.xml . . . 77
themes.xml . . . 78
colors.xml . . . 78
exchange_placeholder.xml . . . 79
ic_launcher_foreground.xml . . . 80
ic_launcher.xml . . . 82
ic_launcher_round.xml . . . 83
ic_launcher.xml . . . 85
ic_launcher_round.xml . . . 86
backup_rules.xml . . . 86
data_extraction_rules.xml . . . 87
proguard-rules.pro . . . 87
.gitignore . . . 88

Аудит SIS3: что выполнено и что нельзя назвать идеальным 91

Практика: изменения, которые ты должен сделать сам 94

Вопросы на защите: отвечай своими словами 97

SIS2: пять решений Kotlin отдельно от Android 100

01_TwoSum.kt . . . 100
14_LongestCommonPrefix.kt . . . 102
20_ValidParentheses.kt . . . 103
21_MergeTwoSortedLists.kt . . . 105
242_ValidAnagram.kt . . . 107

Куда смотреть, если забыл термин 108

Как пользоваться этим разбором

Этот разбор написан для твоего RateRadar KZ, а не для абстрактного учебного приложения. Снимок файлов: 9 октября 2026 года, текущая рабочая папка kbtu_android. В проекте уже были незакоммиченные удаления комментариев в четырёх Kotlin-файлах. Код приложения при аудите не менялся. Номера строк ниже относятся именно к этому снимку.

Цель: ты открываешь файл, понимаешь каждую конструкцию, объясняешь, какую задачу она решает, и можешь сам сделать небольшую правку. Прочитать книгу один раз недостаточно: понимание проверяется тем, можешь ли ты предсказать результат изменения и проверить его в приложении.

Порядок, который помогает не запутаться

1. Сначала главы про Kotlin, Android, Compose и карту приложения. Не пытайся понимать NavHost до того, как разберёшь параметры функций и лямбды. 2. Затем MainActivity, модель и данные, тема, компоненты, экраны, навигация. 3. После этого сценарии нажатий и упражнения. 4. Аудит требований можно прочитать сразу: он отделяет выполненное от спорного.

На первом проходе читай с открытым Android Studio. После маленького раздела закрывай PDF и объясняй идею своими словами. На втором проходе меняй текст, отступ или порядок элементов, запускай приложение, затем возвращай изменение. На третьем проходе пройди пробную защиту в конце книги.

Что здесь означает «весь код»

Разобраны все 14 Kotlin-файлов приложения, все XML-ресурсы, Manifest, Gradle-конфигурация и тест. В приложении к книге разобраны и пять решений SIS2. Скрипты gradlew и gradlew.bat, бинарный gradle-wrapper.jar, сгенерированные build-файлы и внутренние кэши объяснены по назначению: это готовые инструменты, а не написанная тобой логика приложения. Не нужно учить их системную реализацию, чтобы объяснить SIS3.

В разборе кода показаны настоящие строки с номерами. Соседние строки одного вызова объясняются вместе, потому что параметр, открывающая скобка и закрывающая скобка составляют одну мысль. Повторяющиеся импорты тоже разобраны. Пустые строки только разделяют блоки. Очень длинная строка кода в PDF переносится; продолжение не получает нового номера строки.

> Гарантировать оценку или «идеальное соответствие» нельзя: есть установленный пробел в истории Git, спорные формальные пункты дизайна и утверждения AI_USAGE.md, которые можно подтвердить только твоим реальным опытом. Учебник нужен для подготовки; на самой защите, согласно SIS3, AI использовать нельзя.

Что именно уже делает приложение

RateRadar KZ сейчас показывает демонстрационные курсы покупки и продажи USD у 12 обменников. Все записи вручную заданы в SampleData.kt. Цифры не скачиваются из интернета и не меняются от времени. updatedAt - обычная строка вроде «10:30», а не работающий механизм обновления.

На главном экране видны фильтры городов и карточки обменников. Нажатие на карточку открывает детали. Нажатие на сердце меняет избранное. Сердце в верхней панели открывает отдельный список избранного. В Karaganda специально нет обменников: это демонстрация пустого результата фильтра.

В SIS1 описана будущая идея: конвертер, разные валюты, сортировка, сеть и сохранение после перезапуска. SIS3 посвящён экранам, стилю, навигации и простому состоянию. Поэтому отсутствие сети, базы и конвертера здесь не означает автоматическое нарушение SIS3. На защите называй текущую версию демо интерфейса, а будущие возможности - планом.

```
Запуск
  MainActivity
    RateRadarTheme
      RateRadarNavGraph
        ExchangeOfficeListScreen
          OfficeCard -> RateBlock
          EmptyState при пустом фильтре
        OfficeDetailsScreen
          Image, информация, RateBlock, кнопка
          EmptyState при неизвестном id
        FavoritesScreen
          OfficeCard или EmptyState
```

Разница между экраном и Activity

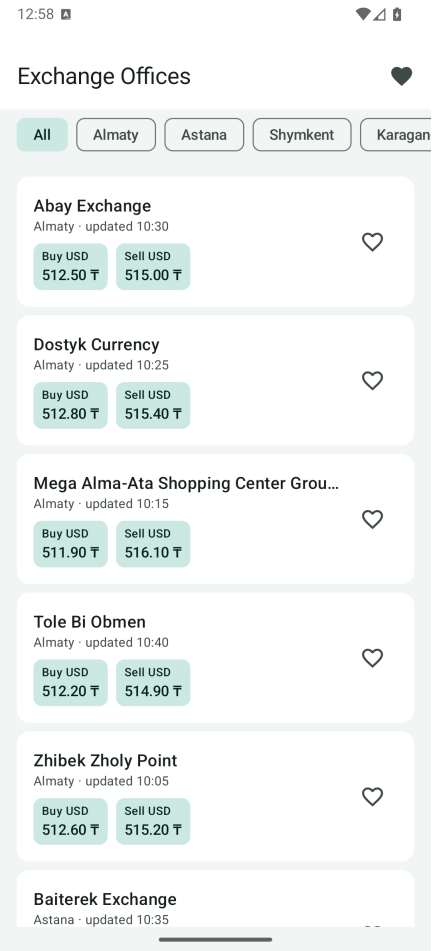
Activity - Android-компонент, который система может создать и показать в окне. Здесь Activity одна: MainActivity. Три экрана - Kotlin-функции внутри этой Activity. При переходе NavHost показывает другую функцию; новая Activity для каждого обменника не создаётся.

Три вида данных

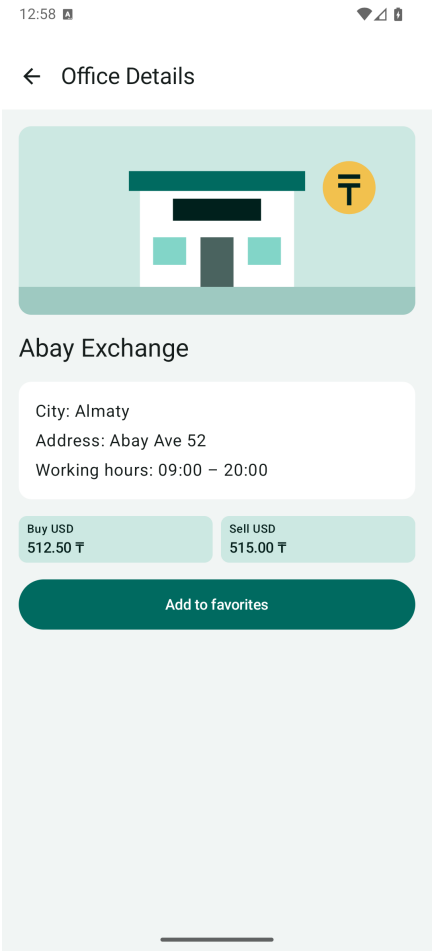
Что	Где	Для чего
Все обменники	SampleData.offices	Постоянный локальный список
Выбранный город	ExchangeOfficeListScreen	Отбор карточек на главном экране
Набор избранных id	RateRadarNavGraph	Общее избранное для всех экранов

ExchangeOffice хранит факты об обменнике. Он не содержит поля isFavorite. Факт избранности вычисляется отдельно: есть ли office.id в favoriteIds. Это позволяет одному набору избранного согласованно менять главный экран, детали и Favorites.

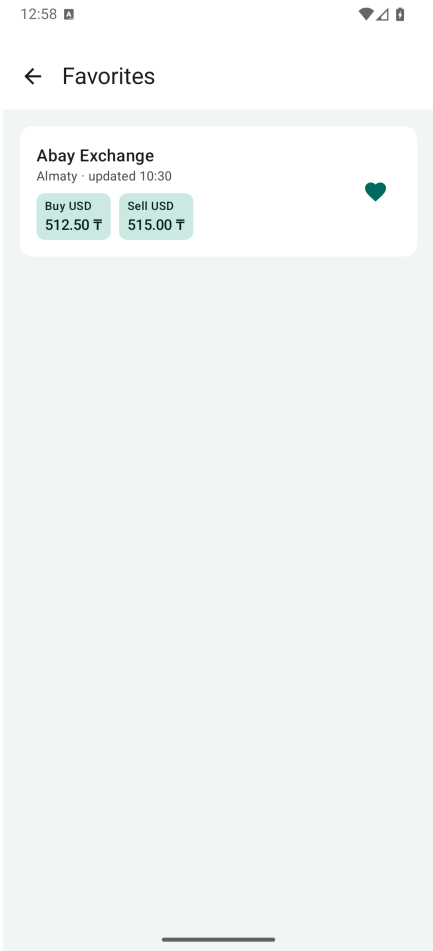
Три экрана, которые ты объясняешь



Список: Scaffold, LazyRow, LazyColumn и OfficeCard



Детали: Scaffold, Column с прокруткой и RateBlock



Избранное: тот же OfficeCard, другой набор записей

Kotlin с нуля: как читать эти файлы

Переменная, значение, тип

```
val city: String = "Almaty"
var selectedCity: String = "All"
selectedCity = "Astana"
```

val объявляет имя, которому нельзя присвоить другое значение. var позволяет новое присваивание. Двоеточие вводит тип. String - текст, Int - целое число, Double - число с дробной частью, Boolean - true или false. Знак = присваивает; == сравнивает. Кавычки создают строку. Имена чувствительны к регистру: city и City - разные имена.

Kotlin часто выводит тип сам: val id = 3 имеет тип Int, val rate = 512.5 - Double. Суффикс f означает Float: 1f нужен weight, который принимает Float. val запрещает переназначить ссылку, но не делает произвольный объект внутри неё неизменяемым. В нашем коде ExchangeOffice содержит val-свойства, а listOf/setOf предоставляют интерфейс коллекции только для чтения.

Функция и вызов

```
fun add(a: Int, b: Int): Int {
    return a + b
}
val result = add(a = 2, b = 3)
```

fun объявляет функцию: именованный кусок работы. a и b - параметры объявления; 2 и 3 - аргументы конкретного вызова. Тип после закрывающей скобки - результат. return завершает функцию с результатом. Вызов без return-типа часто возвращает Unit: полезного значения нет, задача - выполнить действие. Unit не означает null.

Круглые скобки содержат параметры или аргументы. Фигурные скобки - тело функции или лямбды. Запятая разделяет аргументы. Запятая в конце последнего аргумента разрешена: это trailing comma, она упрощает добавление следующего параметра. Точка обращается к члену: office.name, SampleData.offices.

```
fun details(officeId: Int) = "details/$officeId"
```

Это короткая функция с одним выражением. Результат выводится как String. $officeId вставляет значение в строку: details(3) возвращает «details/3». Для более сложного выражения используют ${...}. Не путай фигурные скобки внутри маршрута «details/{officeId}» с интерполяцией: там нет $, это шаблон для навигации.

Именованные аргументы и значения по умолчанию

```
fun show(label: String, count: Int = 1) { }
show(label = "Offices")
```

В объявлении count = 1 задаёт значение по умолчанию. При вызове label = «Offices» передаёт аргумент по имени. Поэтому одинаковый знак = в разных местах читается по-разному. В Compose modifier: Modifier = Modifier позволяет не передавать настройки размера и отступа.

Лямбда: функцию можно передать как значение

```
val action: () -> Unit = { println("clicked") }
action()
val actionWithId: (Int) -> Unit = { id -> println(id) }
actionWithId(3)
```

() -> Unit - тип функции без входных параметров и полезного результата. (Int) -> Unit - функция, которая принимает Int. В { id -> ... } имя id задаёт параметр лямбды; после стрелки идёт её тело. Создать лямбду и выполнить лямбду - разные действия. action хранит функцию, action() её выполняет.

```
IconButton(onClick = onOpenFavorites) {
    Icon(...)
}
```

onClick = onOpenFavorites передаёт функцию, чтобы кнопка вызвала её позже при нажатии. Если написать onOpenFavorites(), ты запустишь действие во время построения интерфейса, а результатом будет Unit вместо требуемой функции. Последняя лямбда может стоять за круглыми скобками. Здесь { Icon(...) } описывает содержимое кнопки, а не реакцию на нажатие.

```
OfficeCard(
    onClick = { onOfficeClick(office.id) },
    onFavoriteClick = { onFavoriteClick(office.id) },
    ...
)
```

Карточке достаточно события без параметров: её внешний код уже знает office. Лямбда запоминает доступ к этому office и при нажатии передаёт его id в обработчик экрана. Такое использование внешней переменной называется захватом. Само наличие лямбды не делает работу асинхронной.

it и именованный параметр

```
SampleData.offices.filter { it.city == selectedCity }
SampleData.offices.filter { office -> office.city == selectedCity }
```

Это две формы одной проверки. У лямбды с одним параметром Kotlin позволяет опустить его объявление; тогда он называется it. it - текущий элемент, который filter проверяет, а не весь список и не глобальная переменная. filter возвращает новый список подходящих объектов. Исходный SampleData.offices не удаляется и не меняется.

Коллекции и обобщённые типы

```
val offices: List<ExchangeOffice> = listOf(...)
val favoriteIds: Set<Int> = setOf(1, 3)
val nothing: Set<Int> = emptySet()
val first = offices[0]
```

List - последовательность элементов: порядок важен, повторения допустимы. Индекс начинается с 0. Set - множество уникальных значений: один id не может появиться дважды. Угловые скобки задают тип содержимого: Set<Int> хранит целые числа. Когда ожидаемый тип неизвестен, setOf<Int>() явно объясняет тип пустого набора.

officeId in favoriteIds проверяет наличие. favoriteIds + officeId возвращает набор с добавленным id, favoriteIds - officeId - набор без id. Это не арифметическое сложение чисел: оператор зависит от типов. isEmpty() сообщает, пуст ли список. find { ... } возвращает первый подходящий объект либо null. filter { ... } возвращает все подходящие объекты, возможно пустой список.

Условие тоже может вернуть значение

```
val offices = if (selectedCity == "All") {
    SampleData.offices
} else {
    SampleData.offices.filter { it.city == selectedCity }
}
```

if выбирает ветку по Boolean. В Kotlin это выражение: результат последнего выражения выбранной ветки записывается в offices. В UI if может также решать, что входит в интерфейс: EmptyState или LazyColumn. Не надо сначала создать оба экрана и затем прятать один.

null, знак вопроса, безопасный доступ

```
val office: ExchangeOffice? = SampleData.offices.find { it.id == officeId }
val officeId = backStackEntry.arguments?.getInt("officeId") ?: 0
```

null означает отсутствие значения. ExchangeOffice? допускает объект или null, ExchangeOffice без ? - только объект. ?. вызывает метод лишь если слева не null; иначе весь доступ даёт null. ?: выбирает правую часть, если слева null. В этой строке 0 - запасной id, а не первый элемент списка: реальные id от 1 до 12.

В OfficeDetailsScreen проверка if (office == null) разделяет случаи. В else Kotlin знает, что office существует, и разрешает office.name без ?. Это smart cast: уточнение типа по выполненной проверке. Не нужно лечить отсутствие объекта оператором !!: он принудительно обещает, что null не будет, и при нарушении обещания вызывает ошибку.

class, data class, object, наследование

class описывает тип объектов. ExchangeOffice(...) создаёт конкретный экземпляр. data class просит Kotlin автоматически добавить полезные операции для данных: equals, hashCode, toString, copy и componentN. val в его параметрах создаёт свойства экземпляра. Обычный параметр без val/var сам по себе свойством не становится.

object SampleData создаёт один доступный объект, без необходимости писать SampleData(). const val - константа времени компиляции, здесь строка «All». private ограничивает доступ файлом для объявлений верхнего уровня. class MainActivity : ComponentActivity() означает наследование от ComponentActivity; () вызывает конструктор родителя. override заменяет разрешённую родителем реализацию метода. super.onCreate(...) вызывает родительский метод.

package, import и аннотации

package задаёт адрес объявлений внутри Kotlin-программы: kz.edu.kbtu.rateradar.ui.components. Папки повторяют его для порядка, но файл .kt может содержать несколько функций. import позволяет писать короткое имя Column вместо полного androidx.compose.foundation.layout.Column. Он не запускает функцию и не скачивает библиотеку; скачивание зависимости делает Gradle.

@Composable, @Preview и @OptIn - аннотации, то есть отметки, которые понимают компилятор или инструменты. Они имеют разное назначение. @Composable разрешает участие функции в Compose-интерфейсе. @Preview просит Android Studio показать образец. @OptIn подтверждает осознанное использование экспериментального API библиотеки. ::class передаёт ссылку на тип аннотации, а не создаёт её объект.

Что означает by в состоянии

```
var favoriteIds by remember { mutableStateOf(setOf<Int>()) }
// Та же идея в более явной записи:
val favoriteIdsState = remember { mutableStateOf(setOf<Int>()) }
val ids = favoriteIdsState.value
favoriteIdsState.value = ids + 3
```

by делегирует чтение и запись объекту состояния. Когда читаешь favoriteIds, читается его value. Когда присваиваешь favoriteIds = ..., записывается value. Импорты getValue и setValue включают эти операции. Это не «автоматическое сохранение в базе», а удобная форма обращения к наблюдаемому состоянию.

Android: что происходит от Run до экрана

Инструменты и части приложения

Android Studio - среда разработки: редактор, запуск и отладка. Kotlin - язык твоего кода. Android SDK - набор API и инструментов Android. Jetpack - семейство библиотек; Jetpack Compose - библиотека для описания UI функциями Kotlin. Material 3 - готовые компоненты и правила оформления. Gradle собирает всё в приложение. Эмулятор запускает Android на компьютере.

SDK - не устройство. API level - номер набора системных возможностей. В этом проекте minSdk = 24 задаёт минимальный Android для установки; compileSdk = 36 - API для компиляции; targetSdk = 36 - версию поведения системы, на которую приложение ориентируется. Они не должны быть одним и тем же числом. Bytecode target 17 относится к Java/Kotlin, а не к Android 17.

Сборка простыми словами

```
Исходники Kotlin + XML + зависимости
  -> компиляция Kotlin/Java
  -> ресурсы и класс R
  -> DEX: код для Android Runtime
  -> APK: устанавливаемый пакет
  -> установка на эмулятор
  -> запуск MainActivity
```

APK содержит скомпилированный код, ресурсы и Manifest. Debug-сборка удобна для разработки и подписывается отладочным ключом. AAB - другой формат распространения через магазин; он не нужен для этого задания. Ошибка Gradle Sync означает проблему настройки/зависимостей; ошибка compile - проблему компиляции; ошибка при запуске - уже runtime. «Проект собрался» и «все требования выполнены» - разные проверки.

Manifest определяет вход

AndroidManifest.xml объявляет MainActivity и фильтр MAIN + LAUNCHER. Поэтому система знает, какую Activity открыть по иконке приложения. exported = true разрешает внешнему компоненту, в том числе launcher, запускать эту Activity. В Manifest нет разрешения INTERNET; в коде нет запросов к серверу.

Жизненный цикл

Система создаёт MainActivity и вызывает onCreate. Затем Activity становится видимой и активной. При уходе в фон система вызывает другие lifecycle-методы. При смене конфигурации, например темы или ориентации, Activity обычно пересоздаётся: старый объект исчезает, новый снова проходит onCreate. Здесь эти другие методы не переопределены: их стандартное поведение обеспечивает базовый класс.

Bundle - контейнер значений, который Android использует, в том числе для восстановления состояния. savedInstanceState: Bundle? в onCreate может быть null при первом запуске. Передача его в super.onCreate нужна базовой инфраструктуре. Она не означает, что favoriteIds сам сохранится: сохранение зависит от способа хранения конкретной переменной.

setContent и край экрана

setContent { ... } подключает Compose как содержимое окна Activity. Внутри устанавливается RateRadarTheme, затем граф навигации. enableEdgeToEdge() разрешает рисовать до краёв окна, в область под системными панелями. Чтобы текст и кнопки не оказались под панелью, Scaffold даёт innerPadding. В проекте его применяют к содержимому каждого экрана.

Compose: как из функций получается интерфейс

Декларативный подход без сложных слов

Ты описываешь, что должно быть на экране для текущих данных: если список пуст, сообщение; иначе карточки. Compose строит интерфейс по этому описанию. При изменении наблюдаемого состояния он может снова выполнить нужные функции с новыми значениями. Это recomposition. Повторный вызов UI-функции не равен перезапуску приложения и не равен пересозданию Activity.

Composition - сохранённое Compose описание и связи между элементами интерфейса. Функции @Composable могут вызывать другие composable-функции. Их нельзя произвольно вызывать из обычной функции вне Compose-контекста. Код в них должен описывать UI; переход navController.navigate должен происходить в обработчике нажатия, а не сам собой при каждом пересчёте.

Контейнеры: в каком порядке лежат элементы

Элемент	Смысл в твоём коде
Column	Дети сверху вниз: заголовок, подзаголовок, курсы
Row	Дети слева направо: информация и сердце; покупка и продажа
Box	Дети могут лежать друг поверх друга; в текущем runtime-коде не используется
LazyColumn	Вертикальный прокручиваемый список карточек
LazyRow	Горизонтальный прокручиваемый список городов

Обычная Column не начинает прокручиваться сама. В деталях используется Column.verticalScroll(rememberScrollState()). В списках LazyColumn/LazyRow уже управляют прокруткой. Lazy-компоненты создают и размещают нужные элементы по мере необходимости, в том числе рядом с видимой областью; формулировка «ровно только видимые» слишком строгая.

Scaffold: что это и что он не делает сам

```
Scaffold(
    topBar = { TopAppBar(title = { Text("Title") }) },
) { innerPadding ->
    Column(modifier = Modifier.padding(innerPadding)) { ... }
}
```

Scaffold - контейнер типовой структуры экрана. У него есть места, или slots, для topBar, bottomBar, floatingActionButton и содержимого. В проекте используется topBar и содержимое. TopAppBar - сама верхняя панель: title задаёт заголовок, navigationIcon - кнопку слева, actions - действия справа.

Scaffold не создаёт навигацию и не делает кнопку Back рабочей сам. Кнопке передают onBackClick; его реализацию определяет NavGraph. innerPadding - вычисленные отступы, которые учитывают панели

Scaffold и системные края. Scaffold передаёт их в лямбду, но ты сам применяешь их к контенту. Если забыть padding(innerPadding), элементы могут попасть под TopAppBar.

Modifier: настройка элемента снаружи

```
Modifier
    .fillMaxWidth()
    .height(Sizes.detailsImageHeight)
    .clip(MaterialTheme.shapes.medium)
```

Modifier - цепочка инструкций для размера, отступов, рисования, ввода и других свойств UI-элемента. Начальное Modifier - пустая цепочка. Каждый вызов возвращает цепочку с дополнительной инструкцией. Модификаторы читают сверху вниз, но окончательное измерение происходит с учётом ограничений родителя; они не рисуют интерфейс построчно как команды на холсте.

fillMaxWidth() просит занять доступную ширину. fillMaxSize() - доступную ширину и высоту. Это не обязательно весь экран телефона: речь о месте, которое выделил родитель. height(180.dp) задаёт желаемую высоту в рамках ограничений. heightIn(min = 48.dp) задаёт минимум, поэтому кнопка может стать выше для текста. size(64.dp) задаёт ширину и высоту иконки.

padding добавляет свободное место вокруг содержимого. .padding(innerPadding).padding(Spacing.large) применяет две оболочки: отступ от панелей и дизайнерский отступ. contentPadding у LazyColumn задаёт отступ внутри прокручиваемого содержимого, а modifier.padding - снаружи самого списка. Arrangement.spacedBy - расстояние между соседними детьми, без внешнего края.

Порядок имеет значение

```
Modifier.background(color).padding(16.dp)
Modifier.padding(16.dp).background(color)
```

Это учебный пример, а не строка из проекта. В первой записи фон покрывает и добавленный отступ; во второй фон рисуется внутри отступа. Поэтому «просто переставить модификаторы» может менять вид. В деталях padding(innerPadding) стоит до verticalScroll, чтобы верхняя панель оставалась вне прокручиваемого содержимого.

weight, alignment, arrangement

В OfficeCard Column.weight(1f) получает оставшуюся ширину Row после места под сердце. Поэтому длинное имя не выталкивает сердце наружу. В деталях два RateBlock.weight(1f) делят доступную ширину Row поровну с учётом расстояния между ними. weight работает в подходящем RowScope или ColumnScope; нельзя бездумно перенести такой вызов в произвольный контейнер.

Alignment отвечает за положение по поперечной оси: в Column horizontalAlignment задаёт горизонталь, в Row verticalAlignment - вертикаль. Arrangement отвечает за распределение по основной оси: verticalArrangement у Column, horizontalArrangement у Row. EmptyState центрирует детей по горизонтали и распределяет их по центру вертикально.

dp и размер текста

dp - единица размера, которая учитывает плотность экрана, чтобы элемент не становился крошечным на плотном дисплее. 48 dp не равно всегда 48 физических пикселей. sp применяется к тексту и дополнительно учитывает пользовательский масштаб шрифта. В проекте свои размеры шрифта не задаются: используются роли MaterialTheme.typography.

Текст, изображения, доступность

Text рисует текст. maxLines = 1 ограничивает количество строк; overflow = TextOverflow.Ellipsis показывает многоточие, когда текст не помещается. Это не изменяет строку office.name в данных. В деталях название сейчас переносится полностью, потому что там этих параметров нет.

Image рисует изображение через painter. painterResource берёт drawable из приложения. Icon удобен для небольших символов ImageVector. contentDescription - описание для технологий доступности, например экранного чтения. Для кнопки Back оно важно; у декоративного рисунка EmptyState оно null, потому что смысл уже написан рядом текстом. null не значит «нужно всегда добавлять произвольное описание».

Card - контейнер карточки с оформлением Material; overload с onClick делает его кликабельным. Surface - более общий оформленный контейнер с цветом и формой. Button - основная кнопка, OutlinedButton - кнопка с контуром, IconButton - кнопка с иконкой. Вложенная IconButton обрабатывает своё нажатие отдельно от onClick карточки; это проверено в работающем приложении.

Состояние: почему экран меняется после нажатия

Три разные задачи одной строки

```
var favoriteIds by remember { mutableStateOf(setOf<Int>()) }
```

mutableStateOf создаёт наблюдаемое значение: Compose узнаёт, что value изменился. remember удерживает тот же объект состояния между пересчётами этой части интерфейса. by позволяет обращаться к value коротким именем. Если убрать remember, создание пустого набора может повторяться при пересчёте. Если использовать обычный var без наблюдаемого состояния, Compose не обязан увидеть присваивание и обновить UI.

Состояние не хранит готовый нарисованный экран. favoriteIds содержит только id, например {1, 3}. От него каждый экран вычисляет свой вид: сердце заполнено или нет, какая кнопка показана, какие записи попали в Favorites. Когда набор меняется, чтения этого состояния позволяют Compose запланировать соответствующее обновление.

Почему присваивается новый Set

```
favoriteIds = if (officeId in favoriteIds) {
    favoriteIds - officeId
} else {
    favoriteIds + officeId
}
```

Было {}, нажали 3: получилось {3}. Нажали 1: {3, 1}. Нажали 3 снова: {1}. + и - создают результат, который присваивается состоянию. Не делай mutableSetOf внутри mutableStateOf и простое .add(): изменение внутреннего ненаблюдаемого объекта само по себе не является записью в state.value, поэтому UI может остаться старым.

Где состояние живёт и почему

Все три экрана используют избранное, поэтому оно находится в их общем родителе RateRadarNavGraph. Это state hoisting: состояние подняли на уровень, где его можно разделить. Данные идут вниз в параметрах, события идут вверх через функции. OfficeCard не знает NavController и не хранит своё отдельное избранное; это делает карточку пригодной и для главного списка, и для Favorites.

Выбранный город нужен только главному экрану, поэтому selectedCity находится в ExchangeOfficeListScreen. Список offices вычисляется из selectedCity. Не нужен второй mutableStateOf для отфильтрованного списка: иначе пришлось бы следить, чтобы город и список всегда менялись согласованно.

remember и rememberSaveable

Событие	remember	rememberSaveable
Recomposition	Обычно сохраняется в той же позиции composition	Сохраняется
Пересоздание Activity	Сбрасывается	Может восстановиться через saved state
Полный новый запуск после force-stop	Сбрасывается	Не является гарантированным постоянным хранилищем
Навигация назад	Зависит от жизненного цикла composition	Navigation Compose может восстановить сохранённое состояние entry

В проекте selectedCity - String, сохраняемая rememberSaveable автоматически. favoriteIds использует обычный remember: сохраняется при переходах в рамках живого графа, но теряется при пересоздании Activity. Переключение системной тёмной темы тоже может пересоздать Activity. Не обещай «избранное всегда переживает поворот». Для постоянных предпочтений понадобится отдельное хранилище; это будущая функциональность, не требование SIS3.

rememberSaveable может восстановить значение после пересоздания процесса системой, если сохранилось соответствующее saved state. Это не равно сохранению в базе на любом перезапуске. В упражнении сначала научись объяснять текущую разницу; не усложняй демо неподготовленным ViewModel или базой.

Файловая иерархия: что за что отвечает

```
kbtu_android/
  app/                          Android-модуль
    build.gradle.kts            настройки сборки модуля
    src/main/
      AndroidManifest.xml       описание Android-компонентов
      java/kz/edu/kbtu/rateradar/
        MainActivity.kt         начало UI
        model/                  форма данных
        data/                   сами демонстрационные данные
        navigation/             маршруты и общее состояние
        ui/components/          повторно используемые детали UI
        ui/screens/             три экрана
        ui/theme/               цвета, стили, размеры
      res/                      строки, рисунки, XML-темы, иконки
    src/test/                   unit-тесты на компьютере
    build/                      результат сборки, не исходники
  design/                       эскизы
  screenshots/                  изображения для README
  docs/                         описание идеи SIS1
  sis2/                         отдельные задачи Kotlin
  gradle/wrapper/               фиксированная версия Gradle
  settings.gradle.kts           модули и репозитории библиотек
  build.gradle.kts              версии плагинов
  gradle.properties             параметры Gradle
  gradlew / gradlew.bat         запуск Gradle macOS/Linux/Windows
  local.properties              локальный путь SDK
  README.md / AI_USAGE.md       документация задания
```

Android Studio в режиме Android группирует manifests, kotlin+java и res. Это удобное представление, а не буквальные папки на диске. Нажми Project, чтобы увидеть реальную структуру. Папка java содержит Kotlin-файлы: историческое имя каталога не заставляет писать их на Java. .kt - Kotlin, .kts - Kotlin-скрипт, .xml - разметка, .md - текст Markdown.

model, data, navigation, ui - выбранная организация проекта, а не обязательные магические папки Android. Они помогают отвечать: «тип обменника - в model, записи - в data, действие перехода - в navigation, вид карточки - в components». Папка domain показана на твоём скриншоте, но текущий обход файлов не обнаружил в ней исходников; она не добавляет приложению невидимую бизнес-логику.

res/values/strings.xml содержит строки, drawable - рисунки и векторные drawable, mipmap - иконку приложения, values/themes.xml - системную XML-тему окна, values-night/themes.xml - её ночной вариант. ui/theme/Theme.kt задаёт отдельную Compose-тему. Сходство названий не делает это одним файлом: XML настраивает окно Activity, Kotlin-тема - Material-компоненты внутри Compose.

R - автоматически созданный класс идентификаторов ресурсов. R.string.buy означает id строкового ресурса, а не готовую строку. stringResource читает её для текущих ресурсов и языка. BuildConfig также генерируется при сборке; его не надо создавать руками. Кэши .gradle, .kotlin, .idea, build и local.properties не являются учебной логикой и обычно не коммитятся. Не публикуй локальные пути или ключи подписи.

Данные и точка входа: разбор файлов

Далее идёт код с настоящими номерами строк и объяснениями. Сначала модель и данные: они дают смысл параметрам UI. Потом MainActivity: она соединяет тему с навигацией. Для каждого файла начало package и импорты показаны и объяснены отдельно; закрывающие скобки разбираются как границы соответствующего блока.

ExchangeOffice.kt

Путь: app/src/main/java/kz/edu/kbtu/rateradar/model/ExchangeOffice.kt

Это форма одной записи. Вызов ExchangeOffice(...) создаёт одну запись с восемью свойствами. Без отдельной логики сеть, валидация и автогенерация id не появляются.

```
1  package kz.edu.kbtu.rateradar.model
```

Строка 1: package задаёт адрес объявлений: kz.edu.kbtu.rateradar.model. Это пространство имён, не код запуска.

```
3  data class ExchangeOffice(
```

Строка 3: data class объявляет тип ExchangeOffice. Круглая скобка начинает основной конструктор, а не тело функции.

```
4      val id: Int,
```

Строка 4: id - Int-свойство экземпляра. В демоданных это уникальное число, которым пользуются маршрут и избранное.

```
5      val name: String,
```

Строка 5: name - текст названия; val запрещает переустановить его у этого экземпляра.

```
6      val city: String,
```

Строка 6: city - город. Строка должна совпадать со значением фильтра, например Almaty.

```
7      val address: String,
```

Строка 7: address - адрес. Никакой отдельной геолокации или карты здесь нет.

```
8      val workingHours: String,
```

Строка 8: workingHours - текст часов работы; программа не вычисляет, открыт ли обменник сейчас.

```
9      val usdBuy: Double,
```

Строка 9: usdBuy - курс, по которому обменник покупает USD у клиента, задан как Double.

```
10      val usdSell: Double,
```

Строка 10: usdSell - курс, по которому обменник продаёт USD клиенту.

```
11      val updatedAt: String,
12  )
```

Строки 11-12: updatedAt - текст времени обновления. Закрывается список параметров; фигурного тела у класса нет, оно не нужно для одних свойств.

SampleData.kt

Путь: app/src/main/java/kz/edu/kbtu/rateradar/data/SampleData.kt

В каждом вызове восемь значений: id, name, city, address, workingHours, usdBuy, usdSell, updatedAt. Они соответствуют восьми свойствам модели.

```
1  package kz.edu.kbtu.rateradar.data
2
3  import kz.edu.kbtu.rateradar.model.ExchangeOffice
```

Строка 1: package задаёт адрес объявлений: kz.edu.kbtu.rateradar.data. Это пространство имён, не код запуска.

Строка 3: ExchangeOffice. Твоя data class записи обменника.

```
5  // Demo data: the app has no internet, all offices are written here by hand
6  object SampleData {
```

Строки 5-6: Комментарий поясняет локальность данных. object создаёт единый SampleData, к которому обращаются без конструктора.

```
7
8      const val ALL_CITIES = "All"
```

Строки 7-8: ALL_CITIES - строковая константа «All». Одна и та же строка используется для начального фильтра и специальной проверки.

```
9
10     // Karaganda has no offices on purpose, so the filter can show an empty state
11     val cities = listOf(ALL_CITIES, "Almaty", "Astana", "Shymkent", "Karaganda")
```

Строки 9-11: Комментарий объясняет отсутствие записей в Karaganda. listOf создаёт List<String>: All и четыре города. Порядок определяет порядок chip.

```
12
13     val offices = listOf(
14         ExchangeOffice(1, "Abay Exchange", "Almaty", "Abay Ave 52", "09:00 – 20:00",
    512.50, 515.00, "10:30"),
15         ExchangeOffice(2, "Dostyk Currency", "Almaty", "Dostyk Ave 85", "10:00 –
    21:00", 512.80, 515.40, "10:25"),
```

Строки 12-15: offices - List<ExchangeOffice>. Первые два вызова создают id 1 и 2. Позиционные аргументы читаются в порядке свойств модели: id, name, city, address, workingHours, usdBuy, usdSell, updatedAt.

```
16          ExchangeOffice(
17              3,
18              "Mega Alma-Ata Shopping Center Ground Floor Currency Exchange Office",
19              "Almaty",
20              "Rozybakiev St 247A",
21              "10:00 – 22:00",
22              511.90,
23              516.10,
24              "10:15",
25          ),
```

Строки 16-25: Третий обменник специально записан многострочно: id 3, очень длинное name, Almaty, адрес, часы, покупка 511.90, продажа 516.10, обновление 10:15. Это проверка длинного текста; закрывающая), завершает один вызов ExchangeOffice внутри listOf.

```
26          ExchangeOffice(4, "Tole Bi Obmen", "Almaty", "Tole Bi St 101", "09:00 –
    19:00", 512.20, 514.90, "10:40"),
27          ExchangeOffice(5, "Zhibek Zholy Point", "Almaty", "Zhibek Zholy Ave 64",
    "09:30 – 20:30", 512.60, 515.20, "10:05"),
```

Строки 26-27: Записи id 4 и 5, оба Almaty. Они увеличивают число элементов и дают несколько карточек одного города.

```
28          ExchangeOffice(6, "Baiterek Exchange", "Astana", "Nurzhol Blvd 14", "09:00 –
    21:00", 513.00, 515.80, "10:35"),
29          ExchangeOffice(7, "Khan Shatyr Currency", "Astana", "Turan Ave 37", "10:00 –
    22:00", 512.40, 516.00, "10:20"),
30          ExchangeOffice(8, "Kenesary Obmen", "Astana", "Kenesary St 40", "09:00 –
    19:00", 512.90, 515.30, "09:55"),
31          ExchangeOffice(9, "Expo Money Point", "Astana", "Mangilik El Ave 55", "08:00 –
     20:00", 512.70, 515.60, "10:10"),
```

Строки 28-31: Четыре записи Astana с id 6-9. filter по городу вернёт именно эти объекты, не создавая новые id.

```
32          ExchangeOffice(10, "Tauke Khan Exchange", "Shymkent", "Tauke Khan Ave 23",
    "09:00 – 20:00", 511.80, 514.70, "10:00"),
33          ExchangeOffice(11, "Shymkent Plaza Obmen", "Shymkent", "Kunaev Blvd 9", "10:00
     – 21:00", 512.10, 515.10, "10:45"),
34          ExchangeOffice(12, "Respublika Currency", "Shymkent", "Respublika Ave 6",
    "09:00 – 18:00", 511.60, 514.50, "09:50"),
```

Строки 32-34: Три записи Shymkent с id 10-12. Значения Double - демонстрационные константы; 512.10 хранит число, оформление двух знаков после точки задаёт strings.xml.

```
35      )
36  }
```

Строки 35-36: Закрывается listOf и затем object. Данные не меняются при нажатиях: изменяется отдельный Set избранных id.

MainActivity.kt

Путь: app/src/main/java/kz/edu/kbtu/rateradar/MainActivity.kt

Здесь нет самого списка или кнопок. Файл запускает UI и устанавливает общий контекст оформления.

```
1  package kz.edu.kbtu.rateradar
2
3  import android.os.Bundle
4  import androidx.activity.ComponentActivity
5  import androidx.activity.compose.setContent
6  import androidx.activity.enableEdgeToEdge
7  import kz.edu.kbtu.rateradar.navigation.RateRadarNavGraph
8  import kz.edu.kbtu.rateradar.ui.theme.RateRadarTheme
```

Строка 1: package задаёт адрес объявлений: kz.edu.kbtu.rateradar. Это пространство имён, не код запуска.

Строка 3: Bundle. Контейнер сохранённых Android-значений; ? допускает отсутствие.

Строка 4: ComponentActivity. Базовый Activity-класс с поддержкой lifecycle и Compose-интеграции.

Строка 5: setContent. Подключает Compose-содержимое к Activity.

Строка 6: enableEdgeToEdge. Настраивает окно для рисования до системных краёв.

Строка 7: RateRadarNavGraph. Твой граф навигации и владелец favoriteIds.

Строка 8: RateRadarTheme. Твоя Compose-тема.

```
10  class MainActivity : ComponentActivity() {
```

Строка 10: MainActivity - класс Android-экрана, наследник ComponentActivity. Двоеточие задаёт наследование, () вызывает конструктор базового класса.

```
11      override fun onCreate(savedInstanceState: Bundle?) {
```

Строка 11: override переопределяет onCreate, который вызывается системой. savedInstanceState - параметр Bundle? с потенциально сохранённым состоянием.

```
12          super.onCreate(savedInstanceState)
```

Строка 12: super.onCreate вызывает базовую реализацию. Нужна инфраструктуре Activity; не означает сохранение всех твоих var.

```
13          enableEdgeToEdge()
```

Строка 13: enableEdgeToEdge включает рисование до краёв окна.

```
14          setContent {
```

Строка 14: setContent принимает лямбду описания Compose UI. Она начинает корневую composition Activity.

```
15              RateRadarTheme {
16                  RateRadarNavGraph()
```

Строки 15-16: RateRadarTheme оборачивает весь вложенный UI. Внутри вызывается RateRadarNavGraph, который выбирает текущий экран.

```
17              }
18          }
19      }
20  }
```

Строки 17-20: Закрываются: content темы, content setContent, метод onCreate, класс MainActivity. Разное вложение скобок показывает, кто чей родитель.

Тема и размеры: как менять оформление в одном месте

Цвет - значение, например Color(0xFF006A60). Цветовая роль - назначение: primary, surface, onSurface. Экрану лучше знать роль, чем конкретный код цвета. Тогда light/dark меняет роль централизованно без переписывания каждого Text.

0x означает шестнадцатеричное число. FF в начале восьмизначного ARGB - непрозрачность, следующие пары - красный, зелёный и синий каналы. onPrimary - цвет содержимого поверх primary, а не «событие нажатия». Имена onOfficeClick - события из нашего соглашения, onSurface - совсем другая система цветовых ролей.

Color.kt хранит конкретные значения. Theme.kt связывает их с Material-ролями и выбирает light или dark по системной теме. MaterialTheme передаёт значения вложенному UI. Type.kt пока оставляет стандартную типографику Material 3. Dimens.kt даёт несколько общих отступов и фиксированных размеров.

Не все ColorScheme-роли переопределены: error, tertiary и новые surfaceContainer-роли получают библиотечные значения по умолчанию. Для данного задания полная ручная замена всех ролей не обязательна, но неверно говорить «мы задали абсолютно все цвета Material». Изменение TealPrimary меняет элементы, использующие primary в светлой теме, а не каждый пиксель рисунка и не TealPrimaryDark.

Dimens.kt

Путь: app/src/main/java/kz/edu/kbtu/rateradar/ui/theme/Dimens.kt

Одинаковое значение в разных местах не всегда означает одну настройку: ширина и отступ - разные задачи. Здесь они разделены двумя object.

```
1  package kz.edu.kbtu.rateradar.ui.theme
2
3  import androidx.compose.ui.unit.dp
```

Строка 1: package задаёт адрес объявлений: kz.edu.kbtu.rateradar.ui.theme. Это пространство имён, не код запуска.

Строка 3: dp. Extension для размера в density-independent pixels.

```
5  // The only spacing values used in the app
6  object Spacing {
```

Строки 5-6: Комментарий и object Spacing: единое место шкалы отступов.

```
7      val small = 4.dp
```

Строка 7: small = 4.dp - небольшой отступ. dp превращает число в Dp.

```
8      val medium = 8.dp
```

Строка 8: medium = 8.dp - промежутки карточек, chip, блоков курсов.

```
 9      val large = 16.dp
```

Строка 9: large = 16.dp - основной внешний/внутренний отступ экранов и карточек.

```
10      val extraLarge = 24.dp
11  }
```

Строки 10-11: extraLarge = 24.dp используется EmptyState. Закрывается Spacing.

```
13  // Other fixed sizes
14  object Sizes {
```

Строки 13-14: object Sizes отдельно описывает размеры объектов, не шкалу промежутков.

```
15      val minTouchTarget = 48.dp
```

Строка 15: minTouchTarget = 48.dp - минимум высоты большой кнопки в Details.

```
16      val emptyStateIcon = 64.dp
```

Строка 16: emptyStateIcon = 64.dp - размер декоративной иконки сообщения.

```
17      val detailsImageHeight = 180.dp
18  }
```

Строки 17-18: detailsImageHeight = 180.dp - высота изображения деталей; закрывается object Sizes.

Color.kt

Путь: app/src/main/java/kz/edu/kbtu/rateradar/ui/theme/Color.kt

Это конкретные цвета, не UI-функции. По одному имени константы понятно назначение, но реальную роль задаёт Theme.kt.

```
1  package kz.edu.kbtu.rateradar.ui.theme
2
3  import androidx.compose.ui.graphics.Color
```

Строка 1: package задаёт адрес объявлений: kz.edu.kbtu.rateradar.ui.theme. Это пространство имён, не код запуска.

Строка 3: Color. Тип цвета Compose.

```
5  // Light theme colors
```

Строка 5: Комментарий отмечает светлую палитру.

```
6  val TealPrimary = Color(0xFF006A60)
```

Строка 6: TealPrimary - основной акцент светлой темы. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
7  val OnTealPrimary = Color(0xFFFFFFFF)
```

Строка 7: OnTealPrimary - текст/иконка на основном акценте. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
8  val TealContainer = Color(0xFF9EF2E3)
```

Строка 8: TealContainer - фон акцентного контейнера. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
9  val OnTealContainer = Color(0xFF00201C)
```

Строка 9: OnTealContainer - содержимое акцентного контейнера. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
10  val GraySecondary = Color(0xFF4A635F)
```

Строка 10: GraySecondary - вторичный акцент. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
11  val OnGraySecondary = Color(0xFFFFFFFF)
```

Строка 11: OnGraySecondary - содержимое на secondary. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
12  val GrayContainer = Color(0xFFCCE8E2)
```

Строка 12: GrayContainer - фон secondaryContainer; используется RateBlock. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
13  val OnGrayContainer = Color(0xFF05201C)
```

Строка 13: OnGrayContainer - содержимое RateBlock. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
14  val LightBackground = Color(0xFFF1F5F4)
```

Строка 14: LightBackground - фон экрана. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
15  val LightSurface = Color(0xFFFFFFFF)
```

Строка 15: LightSurface - фон светлой карточки. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
16  val DarkText = Color(0xFF161D1C)
```

Строка 16: DarkText - основной тёмный текст. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
17  val LightSurfaceVariant = Color(0xFFDAE5E1)
```

Строка 17: LightSurfaceVariant - вариант поверхности. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
18  val GrayText = Color(0xFF3F4947)
```

Строка 18: GrayText - приглушённый текст. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
19  val LightOutline = Color(0xFF6F7977)
```

Строка 19: LightOutline - цвет контуров. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
21  // Dark theme colors
```

Строка 21: Комментарий отмечает тёмную палитру.

```
22  val TealPrimaryDark = Color(0xFF82D5C8)
```

Строка 22: TealPrimaryDark - основной акцент тёмной темы. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
23  val OnTealPrimaryDark = Color(0xFF003731)
```

Строка 23: OnTealPrimaryDark - содержимое тёмного primary. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
24  val TealContainerDark = Color(0xFF005048)
```

Строка 24: TealContainerDark - тёмный primaryContainer. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
25  val OnTealContainerDark = Color(0xFF9EF2E4)
```

Строка 25: OnTealContainerDark - содержимое тёмного primaryContainer. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
26  val GraySecondaryDark = Color(0xFFB1CCC6)
```

Строка 26: GraySecondaryDark - вторичный акцент тёмной темы. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
27  val OnGraySecondaryDark = Color(0xFF1C3531)
```

Строка 27: OnGraySecondaryDark - содержимое тёмного secondary. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
28  val GrayContainerDark = Color(0xFF334B47)
```

Строка 28: GrayContainerDark - тёмный secondaryContainer; фон RateBlock. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
29  val OnGrayContainerDark = Color(0xFFCCE8E2)
```

Строка 29: OnGrayContainerDark - текст на тёмном RateBlock. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
30  val DarkBackground = Color(0xFF0E1514)
```

Строка 30: DarkBackground - тёмный фон экрана. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
31  val DarkSurface = Color(0xFF1A2120)
```

Строка 31: DarkSurface - тёмный фон карточки. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
32  val LightText = Color(0xFFDDE4E1)
```

Строка 32: LightText - светлый основной текст. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
33  val DarkSurfaceVariant = Color(0xFF3F4947)
```

Строка 33: DarkSurfaceVariant - вариант тёмной поверхности. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
34  val LightGrayText = Color(0xFFBEC9C6)
```

Строка 34: LightGrayText - приглушённый светлый текст. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

```
35  val DarkOutline = Color(0xFF899390)
```

Строка 35: DarkOutline - контур тёмной темы. val создаёт цветовое значение; оно будет привязано к роли в Theme.kt. ARGB: FF означает непрозрачность.

Theme.kt

Путь: app/src/main/java/kz/edu/kbtu/rateradar/ui/theme/Theme.kt

Тема выбирает роли. isSystemInDarkTheme читает состояние системы; отдельной кнопки переключения темы в приложении нет.

```
1  package kz.edu.kbtu.rateradar.ui.theme
2
3  import androidx.compose.foundation.isSystemInDarkTheme
4  import androidx.compose.material3.MaterialTheme
5  import androidx.compose.material3.darkColorScheme
6  import androidx.compose.material3.lightColorScheme
7  import androidx.compose.runtime.Composable
```

Строка 1: package задаёт адрес объявлений: kz.edu.kbtu.rateradar.ui.theme. Это пространство имён, не код запуска.

Строка 3: isSystemInDarkTheme. Читает, выбрана ли системная тёмная тема.

Строка 4: MaterialTheme. Доступ к текущей схеме цветов, typography и shapes.

Строка 5: darkColorScheme. Создаёт тёмную схему с указанными и стандартными ролями.

Строка 6: lightColorScheme. Создаёт светлую схему с указанными и стандартными ролями.

Строка 7: Composable. Аннотация участия функции в Compose.

```
9  private val LightColors = lightColorScheme(
```

Строка 9: private val LightColors - схема доступна только внутри этого файла. lightColorScheme принимает именованные роли.

```
10      primary = TealPrimary,
11      onPrimary = OnTealPrimary,
12      primaryContainer = TealContainer,
13      onPrimaryContainer = OnTealContainer,
```

Строки 10-13: primary получает TealPrimary; onPrimary - цвет над ним; primaryContainer и onPrimaryContainer - фон и текст акцентного контейнера.

```
14      secondary = GraySecondary,
15      onSecondary = OnGraySecondary,
16      secondaryContainer = GrayContainer,
17      onSecondaryContainer = OnGrayContainer,
```

Строки 14-17: secondary и его on-цвет, secondaryContainer и его on-цвет. RateBlock читает последнюю пару.

```
18      background = LightBackground,
19      onBackground = DarkText,
20      surface = LightSurface,
21      onSurface = DarkText,
```

Строки 18-21: background/onBackground - экран и текст на нём; surface/onSurface - карточка и основной текст в ней.

```
22      surfaceVariant = LightSurfaceVariant,
23      onSurfaceVariant = GrayText,
24      outline = LightOutline,
25  )
```

Строки 22-25: surfaceVariant/onSurfaceVariant - вариант поверхности и вторичный текст; outline - контур. Закрывается создание LightColors.

```
27  private val DarkColors = darkColorScheme(
```

Строка 27: DarkColors создаётся той же формой, но с darkColorScheme. private имеет то же ограничение доступа.

```
28      primary = TealPrimaryDark,
29      onPrimary = OnTealPrimaryDark,
30      primaryContainer = TealContainerDark,
31      onPrimaryContainer = OnTealContainerDark,
```

Строки 28-31: Акцентные роли тёмной схемы получают соответствующие константы с суффиксом Dark.

```
32      secondary = GraySecondaryDark,
33      onSecondary = OnGraySecondaryDark,
34      secondaryContainer = GrayContainerDark,
35      onSecondaryContainer = OnGrayContainerDark,
```

Строки 32-35: Вторичные роли тёмной схемы: secondary и container с содержимым.

```
36      background = DarkBackground,
37      onBackground = LightText,
38      surface = DarkSurface,
39      onSurface = LightText,
```

Строки 36-39: Тёмные фон и поверхность; содержимое LightText. Имена onBackground и onSurface те же, сами значения другие.

```
40      surfaceVariant = DarkSurfaceVariant,
41      onSurfaceVariant = LightGrayText,
42      outline = DarkOutline,
43  )
```

Строки 40-43: Вариант тёмной поверхности, приглушённый текст, контур. Остальные не переданные роли имеют defaults Material.

```
45  @Composable
46  fun RateRadarTheme(
```

Строки 45-46: Тема - composable-функция-обёртка, поэтому может предоставлять контекст вложенному UI.

```
47      darkTheme: Boolean = isSystemInDarkTheme(),
```

Строка 47: darkTheme по умолчанию читает системную тему. Его можно явно передать для примера или теста.

```
48      content: @Composable () -> Unit,
49  ) {
```

Строки 48-49: content - параметр-функция @Composable ()->Unit. Он описывает вложенный интерфейс; аннотация у типа разрешает composable-вызовы внутри.

```
50      val colors = if (darkTheme) DarkColors else LightColors
```

Строка 50: Условие выбирает готовую DarkColors или LightColors. Обе созданы вне функции, не строятся заново в её теле.

```
52      MaterialTheme(
53          colorScheme = colors,
54          typography = Typography,
55          content = content,
56      )
```

Строки 52-56: MaterialTheme получает выбранную схему, локальный объект Typography и переданный content. content = content передаёт функцию, а не запускает её обычным вызовом.

```
57  }
```

Строка 57: Закрывается RateRadarTheme. Тема действует на UI внутри её content, а не на все приложения телефона.

Type.kt

Путь: app/src/main/java/kz/edu/kbtu/rateradar/ui/theme/Type.kt

Одна строка конфигурации: экраны получают стандартные titleMedium/bodyLarge и другие роли.

```
1  package kz.edu.kbtu.rateradar.ui.theme
2
3  import androidx.compose.material3.Typography
```

Строка 1: package задаёт адрес объявлений: kz.edu.kbtu.rateradar.ui.theme. Это пространство имён, не код запуска.

Строка 3: Typography. Тип типографики Material 3; позже локальный val с таким именем.

```
5  val Typography = Typography()
```

Строка 5: val Typography = Typography() создаёт типографику с библиотечными стандартными размерами и весами. Слева имя твоего значения, справа вызов конструктора импортированного класса. Набор не содержит кастомного шрифта.

Переиспользуемые компоненты: разбираем внутренности

Компонент - функция UI, которую удобно вызывать с разными данными. Параметры задают то, что меняется от места к месту. OfficeCard получает объект и callbacks; RateBlock - подпись и курс; EmptyState - иконку, заголовок и пояснение. Они повторно используются в настоящих экранах, а не только в Preview.

Параметр modifier передаётся снаружи в корневой элемент. Так экран решает, что карточка занимает всю ширину, а компонент решает, что внутри неё Row, Column и сердце. Если игнорировать входной modifier и поставить новый Modifier, родительские настройки потеряются.

RateBlock.kt

Путь: app/src/main/java/kz/edu/kbtu/rateradar/ui/components/RateBlock.kt

Используется внутри OfficeCard и напрямую в Details. Дизайн блока один, данные и ширина различаются.

```
 1  package kz.edu.kbtu.rateradar.ui.components
 2
 3  import android.content.res.Configuration
 4  import androidx.compose.foundation.layout.Column
 5  import androidx.compose.foundation.layout.padding
 6  import androidx.compose.material3.MaterialTheme
 7  import androidx.compose.material3.Surface
 8  import androidx.compose.material3.Text
 9  import androidx.compose.runtime.Composable
10  import androidx.compose.ui.Modifier
11  import androidx.compose.ui.res.stringResource
12  import androidx.compose.ui.tooling.preview.Preview
13  import kz.edu.kbtu.rateradar.R
14  import kz.edu.kbtu.rateradar.ui.theme.RateRadarTheme
15  import kz.edu.kbtu.rateradar.ui.theme.Spacing
```

Строка 1: package задаёт адрес объявлений: kz.edu.kbtu.rateradar.ui.components. Это пространство имён, не код запуска.

Строка 3: Configuration. Константа UI_MODE_NIGHT_YES для тёмного Preview.

Строка 4: Column. Вертикальное расположение детей.

Строка 5: padding. Модификатор отступов; импорт extension-функции.

Строка 6: MaterialTheme. Доступ к текущей схеме цветов, typography и shapes.

Строка 7: Surface. Оформленный контейнер с фоном и формой.

Строка 8: Text. Composable текста.

Строка 9: Composable. Аннотация участия функции в Compose.

Строка 10: Modifier. Цепочка настроек UI-элемента.

Строка 11: stringResource. Читает строковый ресурс и подставляет аргументы.

Строка 12: Preview. Аннотация образца для редактора Android Studio.

Строка 13: R. Сгенерированные идентификаторы Android-ресурсов.

Строка 14: RateRadarTheme. Твоя Compose-тема.

Строка 15: Spacing. Твои общие отступы 4/8/16/24 dp.

```
17  // Small box with a label ("Buy USD") and a rate ("512.50 □")
18  @Composable
19  fun RateBlock(
```

Строки 17-19: Комментарий описывает блок. @Composable + fun объявляют повторно используемую UI-функцию.

```
20      label: String,
```

Строка 20: label - произвольная подпись String, например Buy USD. Компонент не жёстко привязан к одной подписи.

```
21      rate: Double,
```

Строка 21: rate - Double-значение курса.

```
22      modifier: Modifier = Modifier,
23  ) {
```

Строки 22-23: modifier - настройка снаружи; по умолчанию пустой Modifier. Закрывается объявление параметров и начинается тело.

```
24      Surface(
25          modifier = modifier,
26          color = MaterialTheme.colorScheme.secondaryContainer,
27          shape = MaterialTheme.shapes.small,
28      ) {
```

Строки 24-28: Surface получает входной modifier, фон secondaryContainer и форму small. Его последний content-блок начинается после) {.

```
29          Column(modifier = Modifier.padding(horizontal = Spacing.medium, vertical =
    Spacing.small)) {
```

Строка 29: Column ставит два Text вертикально. Внутренний padding: 8 dp по горизонтали, 4 dp по вертикали.

```
30              Text(
31                  text = label,
32                  style = MaterialTheme.typography.labelSmall,
33                  color = MaterialTheme.colorScheme.onSecondaryContainer,
34              )
```

Строки 30-34: Первый Text рисует label с labelSmall и onSecondaryContainer - цветом текста над выбранным фоном.

```
35              Text(
36                  text = stringResource(R.string.rate_value, rate),
37                  style = MaterialTheme.typography.titleSmall,
38                  color = MaterialTheme.colorScheme.onSecondaryContainer,
39              )
```

Строки 35-39: Второй Text форматирует rate через R.string.rate_value: два знака после десятичного разделителя и знак тенге. titleSmall делает число заметнее.

```
40          }
41      }
42  }
```

Строки 40-42: Закрываются Column, Surface и функция RateBlock.

```
44  @Preview(showBackground = true)
45  @Composable
46  private fun RateBlockPreview() {
47      RateRadarTheme {
48          RateBlock(label = "Buy USD", rate = 512.5)
49      }
50  }
```

Строки 44-50: Светлый Preview: private composable вызывает тему и RateBlock(Buy USD, 512.5). Без аргумента modifier берётся значение по умолчанию.

```
52  @Preview(showBackground = true, uiMode = Configuration.UI_MODE_NIGHT_YES)
53  @Composable
54  private fun RateBlockDarkPreview() {
55      RateRadarTheme {
56          RateBlock(label = "Sell USD", rate = 515.0)
57      }
58  }
```

Строки 52-58: Тёмный Preview задаёт uiMode NIGHT_YES. RateRadarTheme читает эту конфигурацию; label Sell USD и rate 515.0 - только образец.

OfficeCard.kt

Путь: app/src/main/java/kz/edu/kbtu/rateradar/ui/components/OfficeCard.kt

Вложенность читается так: Card -> Row -> Column(name, город, Row(rate, rate)) + IconButton. Сердце справа не должно теряться из-за длинного названия.

```
 1  package kz.edu.kbtu.rateradar.ui.components
 2
 3  import android.content.res.Configuration
 4  import androidx.compose.foundation.layout.Arrangement
 5  import androidx.compose.foundation.layout.Column
 6  import androidx.compose.foundation.layout.Row
 7  import androidx.compose.foundation.layout.padding
 8  import androidx.compose.material.icons.Icons
 9  import androidx.compose.material.icons.filled.Favorite
10  import androidx.compose.material.icons.outlined.FavoriteBorder
11  import androidx.compose.material3.Card
12  import androidx.compose.material3.CardDefaults
13  import androidx.compose.material3.Icon
14  import androidx.compose.material3.IconButton
15  import androidx.compose.material3.MaterialTheme
16  import androidx.compose.material3.Text
17  import androidx.compose.runtime.Composable
18  import androidx.compose.ui.Alignment
19  import androidx.compose.ui.Modifier
20  import androidx.compose.ui.res.stringResource
21  import androidx.compose.ui.text.style.TextOverflow
22  import androidx.compose.ui.tooling.preview.Preview
23  import kz.edu.kbtu.rateradar.R
24  import kz.edu.kbtu.rateradar.data.SampleData
25  import kz.edu.kbtu.rateradar.model.ExchangeOffice
26  import kz.edu.kbtu.rateradar.ui.theme.RateRadarTheme
27  import kz.edu.kbtu.rateradar.ui.theme.Spacing
```

Строка 1: package задаёт адрес объявлений: kz.edu.kbtu.rateradar.ui.components. Это пространство имён, не код запуска.

Строка 3: Configuration. Константа UI_MODE_NIGHT_YES для тёмного Preview.

Строка 4: Arrangement. Распределение детей и расстояние между ними.

Строка 5: Column. Вертикальное расположение детей.

Строка 6: Row. Горизонтальное расположение детей.

Строка 7: padding. Модификатор отступов; импорт extension-функции.

Строка 8: Icons. Вход в набор Material-иконок.

Строка 9: Favorite. Готовая заполненная иконка сердца.

Строка 10: FavoriteBorder. Готовая контурная иконка сердца.

Строка 11: Card. Material-карточка, в одном overload принимает onClick.

Строка 12: CardDefaults. Стандартные настройки Card; здесь создаёт CardColors.

Строка 13: Icon. Composable для небольшого символа.

Строка 14: IconButton. Кликабельная кнопка с иконкой и стандартной touch-зоной.

Строка 15: MaterialTheme. Доступ к текущей схеме цветов, typography и shapes.

Строка 16: Text. Composable текста.

Строка 17: Composable. Аннотация участия функции в Compose.

Строка 18: Alignment. Выравнивание детей по поперечной оси.

Строка 19: Modifier. Цепочка настроек UI-элемента.

Строка 20: stringResource. Читает строковый ресурс и подставляет аргументы.

Строка 21: TextOverflow. Политика переполнения текста, здесь Ellipsis.

Строка 22: Preview. Аннотация образца для редактора Android Studio.

Строка 23: R. Сгенерированные идентификаторы Android-ресурсов.

Строка 24: SampleData. Твой object с городами и списком обменников.

Строка 25: ExchangeOffice. Твоя data class записи обменника.

Строка 26: RateRadarTheme. Твоя Compose-тема.

Строка 27: Spacing. Твои общие отступы 4/8/16/24 dp.

```
29  // One exchange office in a list. Used on the main screen and on Favorites.
30  @Composable
31  fun OfficeCard(
```

Строки 29-31: Комментарий, @Composable и имя функции. Карточка используется в двух реальных списках.

```
32      office: ExchangeOffice,
33      isFavorite: Boolean,
```

Строки 32-33: office - вся запись; isFavorite - уже вычисленный Boolean. Здесь не создаётся отдельное состояние избранного.

```
34      onClick: () -> Unit,
35      onFavoriteClick: () -> Unit,
```

Строки 34-35: Две функции без параметров: нажатие карточки и сердца. Действие передаётся родителем.

```
36      modifier: Modifier = Modifier,
37  ) {
```

Строки 36-37: Входной modifier и начало тела функции.

```
38      Card(
39          onClick = onClick,
40          modifier = modifier,
41          colors = CardDefaults.cardColors(containerColor =
    MaterialTheme.colorScheme.surface),
42      ) {
```

Строки 38-42: Card с onClick: кликабельная карточка. modifier передан в её корень; cardColors переопределяет containerColor на тему surface.

```
43          Row(
44              modifier = Modifier.padding(Spacing.large),
45              verticalAlignment = Alignment.CenterVertically,
46          ) {
```

Строки 43-46: Row с внутренним отступом large ставит контент и сердце рядом, центрирует детей по вертикали.

```
47              Column(modifier = Modifier.weight(1f)) {
```

Строка 47: Column.weight(1f) занимает оставшуюся ширину Row рядом с IconButton. Не «1 пиксель»: это коэффициент распределения.

```
48                  Text(
49                      text = office.name,
50                      style = MaterialTheme.typography.titleMedium,
51                      color = MaterialTheme.colorScheme.onSurface,
52                      maxLines = 1,
53                      overflow = TextOverflow.Ellipsis,
54                  )
```

Строки 48-54: Название office.name: стиль titleMedium, цвет onSurface, одна строка, многоточие при переполнении.

```
55                  Text(
56                      text = stringResource(R.string.city_and_time, office.city,
    office.updatedAt),
57                      style = MaterialTheme.typography.bodySmall,
58                      color = MaterialTheme.colorScheme.onSurfaceVariant,
59                      maxLines = 1,
60                      overflow = TextOverflow.Ellipsis,
61                  )
```

Строки 55-61: Подстрока берётся из city_and_time с двумя параметрами, городом и updatedAt. bodySmall и onSurfaceVariant, также одна строка с Ellipsis.

```
62                  Row(
63                      modifier = Modifier.padding(top = Spacing.medium),
64                      horizontalArrangement = Arrangement.spacedBy(Spacing.medium),
65                  ) {
```

Строки 62-65: Вложенный Row ниже текста: padding сверху medium, расстояние между RateBlock medium.

```
66                      RateBlock(label = stringResource(R.string.buy), rate =
    office.usdBuy)
67                      RateBlock(label = stringResource(R.string.sell), rate =
    office.usdSell)
68                  }
```

Строки 66-68: Два RateBlock получают подписи из stringResource и правильные поля usdBuy/usdSell. Закрывается Row курсов.

```
69              }
70              // IconButton is 48 dp by default, so it is easy to tap
71              IconButton(onClick = onFavoriteClick) {
```

Строки 69-71: Закрывается текстовая Column. Комментарий про 48 dp описывает стандартную кнопку; IconButton связывается с onFavoriteClick.

```
72                  if (isFavorite) {
73                      Icon(
74                          imageVector = Icons.Filled.Favorite,
75                          contentDescription =
    stringResource(R.string.remove_from_favorites),
76                          tint = MaterialTheme.colorScheme.primary,
77                      )
```

Строки 72-77: Если isFavorite, рисуется заполненное сердце с описанием Remove from favorites и primary-цветом.

```
78                  } else {
79                      Icon(
80                          imageVector = Icons.Outlined.FavoriteBorder,
81                          contentDescription =
    stringResource(R.string.add_to_favorites),
82                          tint = MaterialTheme.colorScheme.onSurfaceVariant,
83                      )
84                  }
```

Строки 78-84: Иначе контурное сердце, описание Add to favorites, вторичный цвет. Условие выбирает вид; сам toggle находится выше, в NavGraph.

```
85              }
86          }
87      }
88  }
```

Строки 85-88: Закрываются IconButton, внешний Row, Card и OfficeCard.

```
 90  @Preview(showBackground = true)
 91  @Composable
 92  private fun OfficeCardPreview() {
 93      RateRadarTheme {
 94          OfficeCard(
 95              office = SampleData.offices[0],
 96              isFavorite = true,
 97              onClick = {},
 98              onFavoriteClick = {},
 99          )
100      }
101  }
```

Строки 90-101: Светлый Preview: office = первый элемент списка (индекс 0, id 1), избранный, callbacks пустые. {} - функция без действия, не отсутствие параметра.

```
103  @Preview(showBackground = true, uiMode = Configuration.UI_MODE_NIGHT_YES)
104  @Composable
105  private fun OfficeCardLongNameDarkPreview() {
106      RateRadarTheme {
107          OfficeCard(
108              office = SampleData.offices[2],
109              isFavorite = false,
110              onClick = {},
111              onFavoriteClick = {},
112          )
113      }
114  }
```

Строки 103-114: Тёмный Preview берёт элемент с индексом 2, то есть id 3 и очень длинное name. Он не избранный, поэтому тестируется контурное сердце и обрезка имени.

EmptyState.kt

Путь: app/src/main/java/kz/edu/kbtu/rateradar/ui/components/EmptyState.kt

Используется при пустом избранном, пустом фильтре и неизвестном id. Это настоящая повторная реализация одного дизайна с разным содержимым.

```
 1  package kz.edu.kbtu.rateradar.ui.components
 2
 3  import android.content.res.Configuration
 4  import androidx.compose.foundation.layout.Arrangement
 5  import androidx.compose.foundation.layout.Column
 6  import androidx.compose.foundation.layout.fillMaxSize
 7  import androidx.compose.foundation.layout.padding
 8  import androidx.compose.foundation.layout.size
 9  import androidx.compose.material.icons.Icons
10  import androidx.compose.material.icons.outlined.FavoriteBorder
11  import androidx.compose.material3.Icon
12  import androidx.compose.material3.MaterialTheme
13  import androidx.compose.material3.Surface
14  import androidx.compose.material3.Text
15  import androidx.compose.runtime.Composable
16  import androidx.compose.ui.Alignment
17  import androidx.compose.ui.Modifier
18  import androidx.compose.ui.graphics.vector.ImageVector
19  import androidx.compose.ui.text.style.TextAlign
20  import androidx.compose.ui.tooling.preview.Preview
21  import kz.edu.kbtu.rateradar.ui.theme.RateRadarTheme
22  import kz.edu.kbtu.rateradar.ui.theme.Sizes
23  import kz.edu.kbtu.rateradar.ui.theme.Spacing
```

Строка 1: package задаёт адрес объявлений: kz.edu.kbtu.rateradar.ui.components. Это пространство имён, не код запуска.

Строка 3: Configuration. Константа UI_MODE_NIGHT_YES для тёмного Preview.

Строка 4: Arrangement. Распределение детей и расстояние между ними.

Строка 5: Column. Вертикальное расположение детей.

Строка 6: fillMaxSize. Модификатор доступной ширины и высоты.

Строка 7: padding. Модификатор отступов; импорт extension-функции.

Строка 8: size. Модификатор ширины и высоты.

Строка 9: Icons. Вход в набор Material-иконок.

Строка 10: FavoriteBorder. Готовая контурная иконка сердца.

Строка 11: Icon. Composable для небольшого символа.

Строка 12: MaterialTheme. Доступ к текущей схеме цветов, typography и shapes.

Строка 13: Surface. Оформленный контейнер с фоном и формой.

Строка 14: Text. Composable текста.

Строка 15: Composable. Аннотация участия функции в Compose.

Строка 16: Alignment. Выравнивание детей по поперечной оси.

Строка 17: Modifier. Цепочка настроек UI-элемента.

Строка 18: ImageVector. Тип векторной иконки, параметр EmptyState.

Строка 19: TextAlign. Выравнивание строк текста внутри Text.

Строка 20: Preview. Аннотация образца для редактора Android Studio.

Строка 21: RateRadarTheme. Твоя Compose-тема.

Строка 22: Sizes. Твои общие фиксированные размеры.

Строка 23: Spacing. Твои общие отступы 4/8/16/24 dp.

```
25  @Composable
26  fun EmptyState(
```

Строки 25-26: Composable EmptyState описывает единый вид поясняющего сообщения.

```
27      icon: ImageVector,
```

Строка 27: icon - ImageVector, чтобы разные пустые случаи могли показывать разные символы.

```
28      title: String,
29      message: String,
```

Строки 28-29: title и message - заголовок и пояснение. В реальных экранах их передают из strings.xml.

```
30      modifier: Modifier = Modifier,
31  ) {
```

Строки 30-31: modifier позволяет родителю отдать всё доступное место через fillMaxSize.

```
32      Column(
33          modifier = modifier.padding(Spacing.extraLarge),
34          horizontalAlignment = Alignment.CenterHorizontally,
35          verticalArrangement = Arrangement.Center,
36      ) {
```

Строки 32-36: Корневая Column применяет входной modifier и padding 24 dp. Дети центрируются горизонтально, всё содержимое - по вертикали выделенного места.

```
37          Icon(
38              imageVector = icon,
39              contentDescription = null,
40              modifier = Modifier.size(Sizes.emptyStateIcon),
41              tint = MaterialTheme.colorScheme.primary,
42          )
```

Строки 37-42: Icon использует переданный вектор, размер 64 dp и primary. contentDescription = null, потому что она декоративная, а смысл есть в следующих Text.

```
43          Text(
44              text = title,
45              modifier = Modifier.padding(top = Spacing.large),
46              style = MaterialTheme.typography.titleMedium,
47              color = MaterialTheme.colorScheme.onBackground,
48          )
```

Строки 43-48: Заголовок: padding сверху 16 dp, titleMedium и onBackground.

```
49          Text(
50              text = message,
51              modifier = Modifier.padding(top = Spacing.medium),
52              style = MaterialTheme.typography.bodyMedium,
53              color = MaterialTheme.colorScheme.onSurfaceVariant,
54              textAlign = TextAlign.Center,
55          )
```

Строки 49-55: Сообщение: padding сверху 8 dp, bodyMedium, вторичный цвет и центрирование строк текста.

```
56      }
57  }
```

Строки 56-57: Закрываются Column и функция.

```
59  @Preview(showBackground = true)
60  @Composable
61  private fun EmptyStatePreview() {
62      RateRadarTheme {
63          EmptyState(
64              icon = Icons.Outlined.FavoriteBorder,
65              title = "No favorites yet",
66              message = "Tap the heart on an office to save it here.",
67          )
68      }
69  }
```

Строки 59-69: Светлый Preview под темой показывает сообщение No favorites yet. Здесь размер не растянут явно, поэтому итоговый образец зависит от preview bounds.

```
71  @Preview(showBackground = true, uiMode = Configuration.UI_MODE_NIGHT_YES)
72  @Composable
73  private fun EmptyStateDarkPreview() {
74      RateRadarTheme {
75          Surface(color = MaterialTheme.colorScheme.background) {
76              EmptyState(
77                  icon = Icons.Outlined.FavoriteBorder,
78                  title = "No favorites yet",
79                  message = "Tap the heart on an office to save it here.",
80                  modifier = Modifier.fillMaxSize(),
81              )
82          }
83      }
84  }
```

Строки 71-84: Тёмный Preview добавляет Surface с background, а EmptyState передаёт fillMaxSize. Так фон и центрирование явно показываются на всей области образца.

Три экрана: разбор каждого блока

Экран собирает компоненты, но не должен самостоятельно выдумывать новые цвета или отдельное состояние избранного. Главный экран хранит фильтр, Favorites вычисляет подсписок по id, Details ищет запись по аргументу. Каждому экрану передаются только нужные события.

@OptIn нужен в закреплённой версии Material 3, потому что используемый API верхней панели помечен экспериментальным. Это принятие отметки API, а не включение особого Android-режима. Когда библиотека меняется, необходимость opt-in может меняться; текущую версию менять ради этой защиты не требуется.

ExchangeOfficeListScreen.kt

Путь: app/src/main/java/kz/edu/kbtu/rateradar/ui/screens/ExchangeOfficeListScreen.kt

Стартовый экран. Тут есть собственное состояние города, но состояние избранного ему приходит снаружи.

```
 1  package kz.edu.kbtu.rateradar.ui.screens
 2
 3  import android.content.res.Configuration
 4  import androidx.compose.foundation.layout.Arrangement
 5  import androidx.compose.foundation.layout.Column
 6  import androidx.compose.foundation.layout.PaddingValues
 7  import androidx.compose.foundation.layout.fillMaxSize
 8  import androidx.compose.foundation.layout.fillMaxWidth
 9  import androidx.compose.foundation.layout.padding
10  import androidx.compose.foundation.lazy.LazyColumn
11  import androidx.compose.foundation.lazy.LazyRow
12  import androidx.compose.foundation.lazy.items
13  import androidx.compose.material.icons.Icons
14  import androidx.compose.material.icons.filled.Favorite
15  import androidx.compose.material.icons.outlined.LocationOn
16  import androidx.compose.material3.ExperimentalMaterial3Api
17  import androidx.compose.material3.FilterChip
18  import androidx.compose.material3.Icon
19  import androidx.compose.material3.IconButton
20  import androidx.compose.material3.Scaffold
21  import androidx.compose.material3.Text
22  import androidx.compose.material3.TopAppBar
23  import androidx.compose.runtime.Composable
24  import androidx.compose.runtime.getValue
25  import androidx.compose.runtime.mutableStateOf
26  import androidx.compose.runtime.saveable.rememberSaveable
27  import androidx.compose.runtime.setValue
28  import androidx.compose.ui.Modifier
29  import androidx.compose.ui.res.stringResource
30  import androidx.compose.ui.tooling.preview.Preview
31  import kz.edu.kbtu.rateradar.R
32  import kz.edu.kbtu.rateradar.data.SampleData
33  import kz.edu.kbtu.rateradar.ui.components.EmptyState
34  import kz.edu.kbtu.rateradar.ui.components.OfficeCard
35  import kz.edu.kbtu.rateradar.ui.theme.RateRadarTheme
36  import kz.edu.kbtu.rateradar.ui.theme.Spacing
```

Строка 1: package задаёт адрес объявлений: kz.edu.kbtu.rateradar.ui.screens. Это пространство имён, не код запуска.

Строка 3: Configuration. Константа UI_MODE_NIGHT_YES для тёмного Preview.

Строка 4: Arrangement. Распределение детей и расстояние между ними.

Строка 5: Column. Вертикальное расположение детей.

Строка 6: PaddingValues. Набор отступов для содержимого контейнера.

Строка 7: fillMaxSize. Модификатор доступной ширины и высоты.

Строка 8: fillMaxWidth. Модификатор доступной ширины.

Строка 9: padding. Модификатор отступов; импорт extension-функции.

Строка 10: LazyColumn. Вертикальный ленивый прокручиваемый список.

Строка 11: LazyRow. Горизонтальный ленивый прокручиваемый список.

Строка 12: items. Extension для добавления элементов списка в Lazy-контейнер.

Строка 13: Icons. Вход в набор Material-иконок.

Строка 14: Favorite. Готовая заполненная иконка сердца.

Строка 15: LocationOn. Иконка расположения для пустого фильтра.

Строка 16: ExperimentalMaterial3Api. Аннотация API, на которое нужно OptIn в этой версии.

Строка 17: FilterChip. Переключаемый элемент фильтра с selected и onClick.

Строка 18: Icon. Composable для небольшого символа.

Строка 19: IconButton. Кликабельная кнопка с иконкой и стандартной touch-зоной.

Строка 20: Scaffold. Контейнер структуры экрана с topBar и content.

Строка 21: Text. Composable текста.

Строка 22: TopAppBar. Material верхняя панель.

Строка 23: Composable. Аннотация участия функции в Compose.

Строка 24: getValue. Оператор чтения state.value через by.

Строка 25: mutableStateOf. Создаёт наблюдаемый MutableState.

Строка 26: rememberSaveable. Хранит и восстанавливает сохраняемые значения через saved state.

Строка 27: setValue. Оператор записи state.value через by.

Строка 28: Modifier. Цепочка настроек UI-элемента.

Строка 29: stringResource. Читает строковый ресурс и подставляет аргументы.

Строка 30: Preview. Аннотация образца для редактора Android Studio.

Строка 31: R. Сгенерированные идентификаторы Android-ресурсов.

Строка 32: SampleData. Твой object с городами и списком обменников.

Строка 33: EmptyState. Твоё сообщение для пустого состояния.

Строка 34: OfficeCard. Твоя повторно используемая карточка.

Строка 35: RateRadarTheme. Твоя Compose-тема.

Строка 36: Spacing. Твои общие отступы 4/8/16/24 dp.

```
38  @OptIn(ExperimentalMaterial3Api::class)
39  @Composable
40  fun ExchangeOfficeListScreen(
```

Строки 38-40: OptIn разрешает используемый экспериментальный API Material 3. Composable отмечает функцию экрана.

```
41      favoriteIds: Set<Int>,
42      onOfficeClick: (Int) -> Unit,
43      onFavoriteClick: (Int) -> Unit,
44      onOpenFavorites: () -> Unit,
```

Строки 41-44: favoriteIds - готовое общее множество; onOfficeClick/onFavoriteClick принимают id; onOpenFavorites не нуждается в id.

```
45  ) {
46      var selectedCity by rememberSaveable { mutableStateOf(SampleData.ALL_CITIES) }
```

Строки 45-46: Начало тела. selectedCity хранится через rememberSaveable + mutableStateOf; by облегчает чтение/присваивание. Начальный город All.

```
48      val offices = if (selectedCity == SampleData.ALL_CITIES) {
49          SampleData.offices
50      } else {
51          SampleData.offices.filter { it.city == selectedCity }
52      }
```

Строки 48-52: offices вычисляется: если All - исходный список; иначе filter оставляет записи с совпавшим city. Последняя строка выбранной ветки - результат if.

```
54      Scaffold(
55          topBar = {
56              TopAppBar(
57                  title = { Text(stringResource(R.string.title_offices)) },
```

Строки 54-57: Scaffold topBar содержит TopAppBar; title - composable-лямбда с Text из title_offices.

```
58                  actions = {
59                      IconButton(onClick = onOpenFavorites) {
60                          Icon(
61                              imageVector = Icons.Filled.Favorite,
62                              contentDescription =
    stringResource(R.string.open_favorites),
63                          )
64                      }
65                  },
```

Строки 58-65: actions располагаются справа в панели. IconButton вызовет onOpenFavorites; заполненное сердце и описание объясняют его действие.

```
66              )
67          },
68      ) { innerPadding ->
69          Column(modifier = Modifier.padding(innerPadding)) {
```

Строки 66-69: Закрываются TopAppBar и topBar. Последняя лямбда Scaffold получает innerPadding; Column применяет его к контенту.

```
70              // Horizontal list of city filters
71              LazyRow(
72                  contentPadding = PaddingValues(horizontal = Spacing.large),
73                  horizontalArrangement = Arrangement.spacedBy(Spacing.medium),
74              ) {
```

Строки 70-74: Комментарий и LazyRow городов: боковой contentPadding 16 dp, расстояние 8 dp.

```
75                  items(SampleData.cities) { city ->
76                      FilterChip(
77                          selected = city == selectedCity,
78                          onClick = { selectedCity = city },
79                          label = { Text(city) },
80                      )
81                  }
```

Строки 75-81: Для каждой city создаётся FilterChip. selected = city == selectedCity выбирает активный вид; onClick присваивает город; label рисует его текст. Города - данные, а не stringResource.

```
82              }
83
84              if (offices.isEmpty()) {
85                  EmptyState(
86                      icon = Icons.Outlined.LocationOn,
87                      title = stringResource(R.string.empty_city_title),
88                      message = stringResource(R.string.empty_city_message),
89                      modifier = Modifier.fillMaxSize(),
90                  )
```

Строки 82-90: Закрывается LazyRow. Если offices пуст, показывается EmptyState с LocationOn, ресурсными текстами и fillMaxSize в оставшейся области.

```
91              } else {
92                  LazyColumn(
93                      contentPadding = PaddingValues(Spacing.large),
94                      verticalArrangement = Arrangement.spacedBy(Spacing.medium),
95                  ) {
```

Строки 91-95: Иначе LazyColumn: внутренний отступ 16 dp вокруг карточек, расстояние 8 dp между ними.

```
96                      items(offices, key = { it.id }) { office ->
```

Строка 96: items получает актуальный список; key = { it.id } задаёт устойчивую идентичность элементов. it этой лямбды - один ExchangeOffice; office следующей лямбды - запись для рисования карточки.

```
97                          OfficeCard(
98                              office = office,
99                              isFavorite = office.id in favoriteIds,
```

Строки 97-99: OfficeCard получает запись и проверку её id в общем наборе favoriteIds.

```
100                             onClick = { onOfficeClick(office.id) },
101                             onFavoriteClick = { onFavoriteClick(office.id) },
```

Строки 100-101: Callbacks карточки не принимают параметров, поэтому лямбда передаёт office.id в параметризованный callback экрана при нажатии.

```
102                             modifier = Modifier.fillMaxWidth(),
103                         )
104                     }
105                 }
106             }
107         }
108     }
109 }
```

Строки 102-109: fillMaxWidth растягивает карточку; дальше закрываются её вызов, items, LazyColumn, else, Column, Scaffold content и функция.

```
111 @Preview(showBackground = true)
112 @Composable
113 private fun ExchangeOfficeListScreenPreview() {
114     RateRadarTheme {
115         ExchangeOfficeListScreen(
116             favoriteIds = setOf(1, 3),
117             onOfficeClick = {},
118             onFavoriteClick = {},
119             onOpenFavorites = {},
120         )
121     }
122 }
```

Строки 111-122: Светлый Preview передаёт setOf(1,3), пустые функции событий и тему. Он показывает вид, не настоящую навигацию.

```
124 @Preview(showBackground = true, uiMode = Configuration.UI_MODE_NIGHT_YES)
125 @Composable
126 private fun ExchangeOfficeListScreenDarkPreview() {
127     RateRadarTheme {
128         ExchangeOfficeListScreen(
129             favoriteIds = setOf(2),
130             onOfficeClick = {},
131             onFavoriteClick = {},
132             onOpenFavorites = {},
133         )
134     }
135 }
```

Строки 124-135: Тёмный Preview задаёт ночной uiMode, setOf(2), остальные события пустые.

FavoritesScreen.kt

Путь: app/src/main/java/kz/edu/kbtu/rateradar/ui/screens/FavoritesScreen.kt

Список не хранится отдельно и не синхронизируется вручную: он всегда является вычислением по favoriteIds.

```
 1  package kz.edu.kbtu.rateradar.ui.screens
 2
 3  import android.content.res.Configuration
 4  import androidx.compose.foundation.layout.Arrangement
 5  import androidx.compose.foundation.layout.PaddingValues
 6  import androidx.compose.foundation.layout.fillMaxSize
 7  import androidx.compose.foundation.layout.fillMaxWidth
 8  import androidx.compose.foundation.layout.padding
 9  import androidx.compose.foundation.lazy.LazyColumn
10  import androidx.compose.foundation.lazy.items
11  import androidx.compose.material.icons.Icons
12  import androidx.compose.material.icons.automirrored.filled.ArrowBack
13  import androidx.compose.material.icons.outlined.FavoriteBorder
14  import androidx.compose.material3.ExperimentalMaterial3Api
15  import androidx.compose.material3.Icon
16  import androidx.compose.material3.IconButton
17  import androidx.compose.material3.Scaffold
18  import androidx.compose.material3.Text
19  import androidx.compose.material3.TopAppBar
20  import androidx.compose.runtime.Composable
21  import androidx.compose.ui.Modifier
22  import androidx.compose.ui.res.stringResource
23  import androidx.compose.ui.tooling.preview.Preview
24  import kz.edu.kbtu.rateradar.R
25  import kz.edu.kbtu.rateradar.data.SampleData
26  import kz.edu.kbtu.rateradar.ui.components.EmptyState
27  import kz.edu.kbtu.rateradar.ui.components.OfficeCard
28  import kz.edu.kbtu.rateradar.ui.theme.RateRadarTheme
29  import kz.edu.kbtu.rateradar.ui.theme.Spacing
```

Строка 1: package задаёт адрес объявлений: kz.edu.kbtu.rateradar.ui.screens. Это пространство имён, не код запуска.

Строка 3: Configuration. Константа UI_MODE_NIGHT_YES для тёмного Preview.

Строка 4: Arrangement. Распределение детей и расстояние между ними.

Строка 5: PaddingValues. Набор отступов для содержимого контейнера.

Строка 6: fillMaxSize. Модификатор доступной ширины и высоты.

Строка 7: fillMaxWidth. Модификатор доступной ширины.

Строка 8: padding. Модификатор отступов; импорт extension-функции.

Строка 9: LazyColumn. Вертикальный ленивый прокручиваемый список.

Строка 10: items. Extension для добавления элементов списка в Lazy-контейнер.

Строка 11: Icons. Вход в набор Material-иконок.

Строка 12: ArrowBack. Зеркалируемая стрелка назад для направления интерфейса.

Строка 13: FavoriteBorder. Готовая контурная иконка сердца.

Строка 14: ExperimentalMaterial3Api. Аннотация API, на которое нужно OptIn в этой версии.

Строка 15: Icon. Composable для небольшого символа.

Строка 16: IconButton. Кликабельная кнопка с иконкой и стандартной touch-зоной.

Строка 17: Scaffold. Контейнер структуры экрана с topBar и content.

Строка 18: Text. Composable текста.

Строка 19: TopAppBar. Material верхняя панель.

Строка 20: Composable. Аннотация участия функции в Compose.

Строка 21: Modifier. Цепочка настроек UI-элемента.

Строка 22: stringResource. Читает строковый ресурс и подставляет аргументы.

Строка 23: Preview. Аннотация образца для редактора Android Studio.

Строка 24: R. Сгенерированные идентификаторы Android-ресурсов.

Строка 25: SampleData. Твой object с городами и списком обменников.

Строка 26: EmptyState. Твоё сообщение для пустого состояния.

Строка 27: OfficeCard. Твоя повторно используемая карточка.

Строка 28: RateRadarTheme. Твоя Compose-тема.

Строка 29: Spacing. Твои общие отступы 4/8/16/24 dp.

```
31  @OptIn(ExperimentalMaterial3Api::class)
32  @Composable
33  fun FavoritesScreen(
```

Строки 31-33: OptIn + Composable + объявление FavoritesScreen.

```
34      favoriteIds: Set<Int>,
35      onBackClick: () -> Unit,
36      onOfficeClick: (Int) -> Unit,
37      onFavoriteClick: (Int) -> Unit,
```

Строки 34-37: Входной общий набор id и три callbacks: Back, открыть карточку, переключить избранность.

```
38  ) {
39      val favorites = SampleData.offices.filter { it.id in favoriteIds }
```

Строки 38-39: favorites вычисляется filter исходного списка: сохранить только объекты, чьи id находятся в favoriteIds. Порядок остаётся порядком SampleData, не порядком добавления сердец.

```
41      Scaffold(
42          topBar = {
43              TopAppBar(
44                  title = { Text(stringResource(R.string.title_favorites)) },
```

Строки 41-44: Scaffold и TopAppBar с ресурсным title_favorites.

```
45                  navigationIcon = {
46                      IconButton(onClick = onBackClick) {
47                          Icon(
48                              imageVector = Icons.AutoMirrored.Filled.ArrowBack,
49                              contentDescription = stringResource(R.string.back),
50                          )
51                      }
52                  },
```

Строки 45-52: navigationIcon слева содержит рабочую IconButton через onBackClick. AutoMirrored-стрелка поддерживает направление письма; stringResource(back) - описание действия.

```
53              )
54          },
55      ) { innerPadding ->
```

Строки 53-55: Закрывается topBar; content Scaffold получает innerPadding.

```
56          if (favorites.isEmpty()) {
57              EmptyState(
58                  icon = Icons.Outlined.FavoriteBorder,
59                  title = stringResource(R.string.empty_favorites_title),
60                  message = stringResource(R.string.empty_favorites_message),
61                  modifier = Modifier
62                      .padding(innerPadding)
63                      .fillMaxSize(),
64              )
```

Строки 56-64: При пустом списке EmptyState с сердцем и текстами; padding(innerPadding) удерживает его ниже панели, fillMaxSize даёт место для центрирования.

```
65          } else {
66              LazyColumn(
67                  modifier = Modifier.padding(innerPadding),
68                  contentPadding = PaddingValues(Spacing.large),
69                  verticalArrangement = Arrangement.spacedBy(Spacing.medium),
70              ) {
```

Строки 65-70: Иначе LazyColumn применяет внешнее innerPadding и внутреннее 16 dp; расстояние между карточками 8 dp.

```
71                  items(favorites, key = { it.id }) { office ->
72                      OfficeCard(
73                          office = office,
74                          isFavorite = true,
```

Строки 71-74: items(favorites, key=id) рисует OfficeCard. isFavorite = true корректно: список уже отобран по набору избранного.

```
75                          onClick = { onOfficeClick(office.id) },
76                          onFavoriteClick = { onFavoriteClick(office.id) },
77                          modifier = Modifier.fillMaxWidth(),
78                      )
```

Строки 75-78: onClick и onFavoriteClick передают id выбранной записи в события экрана. Карточка занимает ширину списка.

```
79                  }
80              }
81          }
82      }
83  }
```

Строки 79-83: Закрываются items, LazyColumn, else, content Scaffold и функция.

```
85  @Preview(showBackground = true)
86  @Composable
87  private fun FavoritesScreenPreview() {
88      RateRadarTheme {
89          FavoritesScreen(
90              favoriteIds = setOf(1, 3, 6),
91              onBackClick = {},
92              onOfficeClick = {},
93              onFavoriteClick = {},
94          )
95      }
96  }
```

Строки 85-96: Светлый Preview содержит id 1,3,6 и пустые действия. По ним можно посмотреть повторное использование OfficeCard.

```
98  @Preview(showBackground = true, uiMode = Configuration.UI_MODE_NIGHT_YES)
99  @Composable
100 private fun FavoritesScreenEmptyDarkPreview() {
101     RateRadarTheme {
102         FavoritesScreen(
103             favoriteIds = emptySet(),
104             onBackClick = {},
105             onOfficeClick = {},
106             onFavoriteClick = {},
107         )
108     }
109 }
```

Строки 98-109: Тёмный Preview передаёт emptySet; ожидаемый тип Set<Int> выводится из параметра. Проверяется ветка EmptyState.

OfficeDetailsScreen.kt

Путь: app/src/main/java/kz/edu/kbtu/rateradar/ui/screens/OfficeDetailsScreen.kt

Детали получают идентификатор, сами находят локальную запись и показывают её. Они не принимают всю запись через маршрут.

```
 1  package kz.edu.kbtu.rateradar.ui.screens
 2
 3  import android.content.res.Configuration
 4  import androidx.compose.foundation.Image
 5  import androidx.compose.foundation.layout.Arrangement
 6  import androidx.compose.foundation.layout.Column
 7  import androidx.compose.foundation.layout.Row
 8  import androidx.compose.foundation.layout.fillMaxSize
 9  import androidx.compose.foundation.layout.fillMaxWidth
10  import androidx.compose.foundation.layout.height
11  import androidx.compose.foundation.layout.heightIn
12  import androidx.compose.foundation.layout.padding
13  import androidx.compose.foundation.rememberScrollState
14  import androidx.compose.foundation.verticalScroll
15  import androidx.compose.material.icons.Icons
16  import androidx.compose.material.icons.automirrored.filled.ArrowBack
17  import androidx.compose.material.icons.outlined.Info
18  import androidx.compose.material3.Button
19  import androidx.compose.material3.Card
20  import androidx.compose.material3.CardDefaults
21  import androidx.compose.material3.ExperimentalMaterial3Api
22  import androidx.compose.material3.Icon
23  import androidx.compose.material3.IconButton
24  import androidx.compose.material3.MaterialTheme
25  import androidx.compose.material3.OutlinedButton
26  import androidx.compose.material3.Scaffold
27  import androidx.compose.material3.Text
28  import androidx.compose.material3.TopAppBar
29  import androidx.compose.runtime.Composable
30  import androidx.compose.ui.Modifier
31  import androidx.compose.ui.draw.clip
32  import androidx.compose.ui.layout.ContentScale
33  import androidx.compose.ui.res.painterResource
34  import androidx.compose.ui.res.stringResource
35  import androidx.compose.ui.tooling.preview.Preview
36  import kz.edu.kbtu.rateradar.R
37  import kz.edu.kbtu.rateradar.data.SampleData
38  import kz.edu.kbtu.rateradar.ui.components.EmptyState
39  import kz.edu.kbtu.rateradar.ui.components.RateBlock
40  import kz.edu.kbtu.rateradar.ui.theme.RateRadarTheme
41  import kz.edu.kbtu.rateradar.ui.theme.Sizes
42  import kz.edu.kbtu.rateradar.ui.theme.Spacing
```

Строка 1: package задаёт адрес объявлений: kz.edu.kbtu.rateradar.ui.screens. Это пространство имён, не код запуска.

Строка 3: Configuration. Константа UI_MODE_NIGHT_YES для тёмного Preview.

Строка 4: Image. Composable изображения, принимает painter.

Строка 5: Arrangement. Распределение детей и расстояние между ними.

Строка 6: Column. Вертикальное расположение детей.

Строка 7: Row. Горизонтальное расположение детей.

Строка 8: fillMaxSize. Модификатор доступной ширины и высоты.

Строка 9: fillMaxWidth. Модификатор доступной ширины.

Строка 10: height. Модификатор высоты.

Строка 11: heightIn. Модификатор диапазона высоты, здесь минимальной.

Строка 12: padding. Модификатор отступов; импорт extension-функции.

Строка 13: rememberScrollState. Состояние обычной прокрутки для verticalScroll.

Строка 14: verticalScroll. Модификатор обычной вертикальной прокрутки.

Строка 15: Icons. Вход в набор Material-иконок.

Строка 16: ArrowBack. Зеркалируемая стрелка назад для направления интерфейса.

Строка 17: Info. Иконка информационного сообщения.

Строка 18: Button. Основная Material-кнопка.

Строка 19: Card. Material-карточка, в одном overload принимает onClick.

Строка 20: CardDefaults. Стандартные настройки Card; здесь создаёт CardColors.

Строка 21: ExperimentalMaterial3Api. Аннотация API, на которое нужно OptIn в этой версии.

Строка 22: Icon. Composable для небольшого символа.

Строка 23: IconButton. Кликабельная кнопка с иконкой и стандартной touch-зоной.

Строка 24: MaterialTheme. Доступ к текущей схеме цветов, typography и shapes.

Строка 25: OutlinedButton. Material-кнопка с контуром.

Строка 26: Scaffold. Контейнер структуры экрана с topBar и content.

Строка 27: Text. Composable текста.

Строка 28: TopAppBar. Material верхняя панель.

Строка 29: Composable. Аннотация участия функции в Compose.

Строка 30: Modifier. Цепочка настроек UI-элемента.

Строка 31: clip. Модификатор обрезки рисунка по форме.

Строка 32: ContentScale. Правило масштабирования изображения в своей области.

Строка 33: painterResource. Загружает painter рисунка из ресурсов.

Строка 34: stringResource. Читает строковый ресурс и подставляет аргументы.

Строка 35: Preview. Аннотация образца для редактора Android Studio.

Строка 36: R. Сгенерированные идентификаторы Android-ресурсов.

Строка 37: SampleData. Твой object с городами и списком обменников.

Строка 38: EmptyState. Твоё сообщение для пустого состояния.

Строка 39: RateBlock. Твой блок подписи и курса.

Строка 40: RateRadarTheme. Твоя Compose-тема.

Строка 41: Sizes. Твои общие фиксированные размеры.

Строка 42: Spacing. Твои общие отступы 4/8/16/24 dp.

```
44  @OptIn(ExperimentalMaterial3Api::class)
45  @Composable
46  fun OfficeDetailsScreen(
```

Строки 44-46: OptIn и Composable перед функцией Details.

```
47      officeId: Int,
48      isFavorite: Boolean,
49      onBackClick: () -> Unit,
50      onFavoriteClick: (Int) -> Unit,
```

Строки 47-50: officeId - число из навигации; isFavorite - актуальный вид; два callbacks для Back и переключения id.

```
51  ) {
52      val office = SampleData.offices.find { it.id == officeId }
```

Строки 51-52: find ищет запись с совпавшим id, возвращает ExchangeOffice? (может быть null). Это не SampleData.offices[officeId].

```
54      Scaffold(
55          topBar = {
56              TopAppBar(
57                  title = { Text(stringResource(R.string.title_details)) },
```

Строки 54-57: Scaffold с TopAppBar и постоянным названием Office Details.

```
58                  navigationIcon = {
59                      IconButton(onClick = onBackClick) {
60                          Icon(
61                              imageVector = Icons.AutoMirrored.Filled.ArrowBack,
62                              contentDescription = stringResource(R.string.back),
63                          )
64                      }
65                  },
```

Строки 58-65: Кнопка navigationIcon вызывает onBackClick. Иконка зеркалируется при RTL, описание - ресурс Back.

```
66              )
67          },
68      ) { innerPadding ->
```

Строки 66-68: Закрывается topBar; content-лямбда получает innerPadding.

```
69          if (office == null) {
70              EmptyState(
71                  icon = Icons.Outlined.Info,
72                  title = stringResource(R.string.office_not_found_title),
73                  message = stringResource(R.string.office_not_found_message),
74                  modifier = Modifier
75                      .padding(innerPadding)
76                      .fillMaxSize(),
77              )
```

Строки 69-77: Если office == null, EmptyState объясняет неизвестный id. padding удерживает контент ниже панели, fillMaxSize занимает область.

```
78          } else {
79              Column(
80                  modifier = Modifier
81                      .padding(innerPadding)
82                      .verticalScroll(rememberScrollState())
83                      .padding(Spacing.large),
84                  verticalArrangement = Arrangement.spacedBy(Spacing.large),
85              ) {
```

Строки 78-85: Иначе office существует (smart cast). Column получает innerPadding, прокрутку через rememberScrollState, внутренний отступ 16 dp и расстояние 16 dp между крупными блоками.

```
86                  Image(
87                      painter = painterResource(R.drawable.exchange_placeholder),
88                      contentDescription =
    stringResource(R.string.office_image_description, office.name),
```

Строки 86-88: Image загружает вектор exchange_placeholder, описание форматируется именем office.name. Сейчас слово Photo неточно для иллюстрации.

```
89                      modifier = Modifier
90                          .fillMaxWidth()
91                          .height(Sizes.detailsImageHeight)
92                          .clip(MaterialTheme.shapes.medium),
93                      contentScale = ContentScale.Crop,
94                  )
```

Строки 89-94: fillMaxWidth - доступная ширина, height - 180 dp, clip - Material medium shape, Crop масштабирует с обрезкой для заполнения области. Закрывается Image.

```
96                  Text(
97                      text = office.name,
98                      style = MaterialTheme.typography.headlineSmall,
99                      color = MaterialTheme.colorScheme.onBackground,
100                 )
```

Строки 96-100: Text полного name с headlineSmall и onBackground. Нет maxLines/overflow: имя переносится на сколько нужно строк в пределах ширины; экран прокручивается.

```
102                 Card(
103                     modifier = Modifier.fillMaxWidth(),
104                     colors = CardDefaults.cardColors(containerColor =
    MaterialTheme.colorScheme.surface),
105                 ) {
```

Строки 102-105: Card информации растягивается по ширине, surface берётся из темы.

```
106                     Column(
107                         modifier = Modifier.padding(Spacing.large),
108                         verticalArrangement = Arrangement.spacedBy(Spacing.small),
109                     ) {
```

Строки 106-109: Column внутри карточки: отступ 16 dp и расстояние 4 dp между информационными строками.

```
110                         Text(
111                             text = stringResource(R.string.label_city, office.city),
112                             style = MaterialTheme.typography.bodyLarge,
113                             color = MaterialTheme.colorScheme.onSurface,
114                         )
```

Строки 110-114: Text City: ... подставляет office.city, стиль bodyLarge, onSurface.

```
115                         Text(
116                             text = stringResource(R.string.label_address,
    office.address),
117                             style = MaterialTheme.typography.bodyLarge,
118                             color = MaterialTheme.colorScheme.onSurface,
119                         )
```

Строки 115-119: Text Address: ... подставляет office.address, тот же стиль.

```
120                         Text(
121                             text = stringResource(R.string.label_hours,
    office.workingHours),
122                             style = MaterialTheme.typography.bodyLarge,
123                             color = MaterialTheme.colorScheme.onSurface,
124                         )
125                     }
126                 }
```

Строки 120-126: Text Working hours: ... подставляет office.workingHours; закрываются Text, Column информации и Card.

```
128                 Row(horizontalArrangement = Arrangement.spacedBy(Spacing.medium)) {
129                     RateBlock(
130                         label = stringResource(R.string.buy),
131                         rate = office.usdBuy,
132                         modifier = Modifier.weight(1f),
133                     )
```

Строки 128-133: Row с расстоянием 8 dp. Первый RateBlock показывает Buy USD/usdBuy и получает weight(1f).

```
134                     RateBlock(
135                         label = stringResource(R.string.sell),
136                         rate = office.usdSell,
137                         modifier = Modifier.weight(1f),
138                     )
139                 }
```

Строки 134-139: Второй RateBlock показывает Sell USD/usdSell с таким же weight. Два блока делят доступную ширину.

```
141                 val buttonModifier = Modifier
142                     .fillMaxWidth()
143                     .heightIn(min = Sizes.minTouchTarget)
```

Строки 141-143: Локальный buttonModifier - значение цепочки: ширина плюс минимум 48 dp по высоте. val не UI-элемент; его используют оба вида кнопки.

```
144                 if (isFavorite) {
145                     OutlinedButton(onClick = { onFavoriteClick(office.id) }, modifier
    = buttonModifier) {
146                         Text(stringResource(R.string.remove_from_favorites))
147                     }
```

Строки 144-147: Если избранный, OutlinedButton Remove from favorites. При нажатии передаёт office.id в onFavoriteClick.

```
148                 } else {
149                     Button(onClick = { onFavoriteClick(office.id) }, modifier =
    buttonModifier) {
150                         Text(stringResource(R.string.add_to_favorites))
151                     }
152                 }
```

Строки 148-152: Иначе Button Add to favorites вызывает тот же callback с id. Общий toggle определяет добавить или убрать по текущему набору.

```
153             }
154         }
155     }
156 }
```

Строки 153-156: Закрываются Column, else, content Scaffold и функция. Здесь нет return@Scaffold: используется обычное условное ветвление.

```
158 @Preview(showBackground = true)
159 @Composable
160 private fun OfficeDetailsScreenPreview() {
161     RateRadarTheme {
162         OfficeDetailsScreen(
163             officeId = 1,
164             isFavorite = false,
165             onBackClick = {},
166             onFavoriteClick = {},
167         )
168     }
169 }
```

Строки 158-169: Светлый Preview id 1, isFavorite=false: кнопка Add; пустые события.

```
171  @Preview(showBackground = true, uiMode = Configuration.UI_MODE_NIGHT_YES)
172  @Composable
173  private fun OfficeDetailsScreenDarkPreview() {
174      RateRadarTheme {
175          OfficeDetailsScreen(
176              officeId = 3,
177              isFavorite = true,
178              onBackClick = {},
179              onFavoriteClick = {},
180          )
181      }
182  }
```

Строки 171-182: Тёмный Preview id 3, длинное name и isFavorite=true: кнопка Remove. Проверяет другой вид и большой контент.

Навигация: как id доходит до деталей

```
Нажатие на карточку Abay Exchange
  OfficeCard вызывает свой onClick()
  { onOfficeClick(office.id) } передаёт число 1
  NavGraph выполняет navigate(Routes.details(1))
  Routes.details(1) возвращает "details/1"
  NavHost сопоставляет с "details/{officeId}"
  navArgument задаёт тип Int
  backStackEntry.arguments даёт officeId = 1
  OfficeDetailsScreen(officeId = 1, ...)
  find { it.id == 1 } находит Abay Exchange
```

Маршрут - строковый адрес внутри приложения. «details/{officeId}» - шаблон, «details/1» - конкретный адрес. officeId - имя аргумента, которое должно совпасть в шаблоне, navArgument и getInt. Этот id не индекс списка. После фильтра или изменения порядка office с id 6 остаётся тем же объектом, даже если его позиция стала 0.

NavController, NavHost, composable

NavController управляет переходами и историей посещений. rememberNavController создаёт его с поддержкой Compose. NavHost - место, где отображается текущая destination, и объявление графа. composable(route) регистрирует destination. Это не аннотация @Composable: имя похоже, задачи разные.

История посещений - стек

```
Старт:               [offices]
Открыть избранное:   [offices, favorites]
Открыть его карточку:[offices, favorites, details/1]
Back:                [offices, favorites]
Back:                [offices]
```

Стек хранит порядок посещённых мест. Последний добавленный экран закрывается первым. navigate добавляет переход; popBackStack удаляет верхнюю запись и возвращает к предыдущей. Поэтому Back из Details, открытого из Favorites, возвращает в Favorites. navigate("offices") вместо popBackStack создал бы новое посещение главного экрана и другую историю.

Кнопка в TopAppBar вызывает callback из NavGraph. Системный Back обрабатывается библиотекой навигации. На стартовом экране нет собственной стрелки назад. popBackStack возвращает Boolean о результате, но обработчик (()->Unit) использует действие и не требует вернуть этот Boolean наружу.

RateRadarNavGraph.kt

Путь: app/src/main/java/kz/edu/kbtu/rateradar/navigation/RateRadarNavGraph.kt

В этом файле соединяются структура посещений и общее избранное. У него нет ViewModel и постоянного хранилища.

```
 1  package kz.edu.kbtu.rateradar.navigation
 2
 3  import androidx.compose.runtime.Composable
 4  import androidx.compose.runtime.getValue
 5  import androidx.compose.runtime.mutableStateOf
 6  import androidx.compose.runtime.remember
 7  import androidx.compose.runtime.setValue
 8  import androidx.navigation.NavType
 9  import androidx.navigation.compose.NavHost
10  import androidx.navigation.compose.composable
11  import androidx.navigation.compose.rememberNavController
12  import androidx.navigation.navArgument
13  import kz.edu.kbtu.rateradar.ui.screens.ExchangeOfficeListScreen
14  import kz.edu.kbtu.rateradar.ui.screens.FavoritesScreen
15  import kz.edu.kbtu.rateradar.ui.screens.OfficeDetailsScreen
```

Строка 1: package задаёт адрес объявлений: kz.edu.kbtu.rateradar.navigation. Это пространство имён, не код запуска.

Строка 3: Composable. Аннотация участия функции в Compose.

Строка 4: getValue. Оператор чтения state.value через by.

Строка 5: mutableStateOf. Создаёт наблюдаемый MutableState.

Строка 6: remember. Хранит объект между recomposition в этой позиции.

Строка 7: setValue. Оператор записи state.value через by.

Строка 8: NavType. Тип аргумента навигации, здесь IntType.

Строка 9: NavHost. Хост текущего destination и DSL объявления графа.

Строка 10: composable. Регистрирует маршрут в графе; не @Composable.

Строка 11: rememberNavController. Создаёт NavController с поддержкой Compose.

Строка 12: navArgument. Объявляет именованный аргумент маршрута.

Строка 13: ExchangeOfficeListScreen. Твой экран общего списка и фильтра.

Строка 14: FavoritesScreen. Твой экран избранного.

Строка 15: OfficeDetailsScreen. Твой экран деталей по id.

```
17  object Routes {
```

Строка 17: object Routes группирует адреса без необходимости создать экземпляр.

```
18      const val OFFICES = "offices"
19      const val DETAILS = "details/{officeId}"
20      const val FAVORITES = "favorites"
```

Строки 18-20: Три константы: offices - старт, details/{officeId} - шаблон с аргументом, favorites - избранное.

```
22      fun details(officeId: Int) = "details/$officeId"
23  }
```

Строки 22-23: details(officeId) создаёт конкретную строку через $officeId. При 3 получается details/3. Закрывается Routes.

```
25  @Composable
26  fun RateRadarNavGraph() {
27      val navController = rememberNavController()
```

Строки 25-27: Composable-функция графа создаёт/помнит NavController через rememberNavController.

```
29      var favoriteIds by remember { mutableStateOf(setOf<Int>()) }
```

Строка 29: Общее наблюдаемое состояние Set<Int>. Изначально пусто; remember живёт в графе, by читает и пишет state.value.

```
31      fun toggleFavorite(officeId: Int) {
32          favoriteIds = if (officeId in favoriteIds) {
```

Строки 31-32: Локальная обычная функция toggleFavorite принимает id. Проверка in выбирает убрать или добавить.

```
33              favoriteIds - officeId
34          } else {
35              favoriteIds + officeId
36          }
37      }
```

Строки 33-37: Если был - новый набор без id; иначе новый набор с id. Результат if присваивается favoriteIds. Закрывается функция toggle.

```
39      NavHost(navController = navController, startDestination = Routes.OFFICES) {
```

Строка 39: NavHost получает controller и startDestination. Последняя лямбда объявляет граф.

```
41          composable(Routes.OFFICES) {
42              ExchangeOfficeListScreen(
43                  favoriteIds = favoriteIds,
```

Строки 41-43: composable(OFFICES) регистрирует стартовую destination. Screen получает актуальный общий набор.

```
44                  onOfficeClick = { id -> navController.navigate(Routes.details(id)) },
```

Строка 44: onOfficeClick - лямбда с id; navigate получает конкретный details/id.

```
45                  onFavoriteClick = { id -> toggleFavorite(id) },
```

Строка 45: onFavoriteClick передаёт id в локальный toggleFavorite.

```
46                  onOpenFavorites = { navController.navigate(Routes.FAVORITES) },
47              )
48          }
```

Строки 46-48: onOpenFavorites запускает navigate на адрес favorites. Закрывается экранный вызов и destination.

```
50          composable(
51              route = Routes.DETAILS,
52              arguments = listOf(navArgument("officeId") { type = NavType.IntType }),
53          ) { backStackEntry ->
```

Строки 50-53: DETAILS-route объявляет список аргументов: navArgument("officeId") с type = NavType.IntType. content-лямбда получает запись backStackEntry для текущего посещения.

```
54              val officeId = backStackEntry.arguments?.getInt("officeId") ?: 0
```

Строка 54: ?. безопасно читает Bundle arguments. getInt читает аргумент; ?: 0 служит fallback при null arguments. 0 не совпадает ни с одним id демоданных.

```
55              OfficeDetailsScreen(
56                  officeId = officeId,
57                  isFavorite = officeId in favoriteIds,
```

Строки 55-57: Details получает id и актуальный Boolean officeId in favoriteIds.

```
58                  onBackClick = { navController.popBackStack() },
59                  onFavoriteClick = { id -> toggleFavorite(id) },
60              )
```

Строки 58-60: Back вызывает popBackStack; избранное вызывает toggle. Screen не должен знать реализацию этих действий.

```
61          }
```

Строка 61: Закрывается destination деталей.

```
63          composable(Routes.FAVORITES) {
64              FavoritesScreen(
65                  favoriteIds = favoriteIds,
```

Строки 63-65: Регистрируется favorites, экран получает общий набор.

```
66                  onBackClick = { navController.popBackStack() },
67                  onOfficeClick = { id -> navController.navigate(Routes.details(id)) },
68                  onFavoriteClick = { id -> toggleFavorite(id) },
69              )
```

Строки 66-69: Back удаляет верхнюю запись истории; карточка открывает details/id; сердце вызывает тот же toggle. Это обеспечивает согласованность всех экранов.

```
70            }
71        }
72    }
```

Строки 70-72: Закрываются destination Favorites, DSL графа NavHost и RateRadarNavGraph.

Три сценария: проследи данные руками

Сценарий 1. Выбрать Astana

selectedCity изначально «All». LazyRow рисует города. У Astana onClick присваивает selectedCity = city. Наблюдаемое состояние меняется. Главный экран снова получает актуальное значение. if выбирает filter, в список попадают id 6, 7, 8, 9. items строит их карточки. favoriteIds не меняется: фильтр скрывает записи, а не удаляет их из избранного.

Сценарий 2. Добавить обменник из списка

Изначально favoriteIds пуст. У карточки 1 isFavorite = false, поэтому рисуется FavoriteBorder. Нажатие на её IconButton вызывает onFavoriteClick карточки. Лямбда главного экрана передаёт 1 в onFavoriteClick экрана. NavGraph вызывает toggleFavorite(1). Так как 1 ещё нет, записывается набор {1}. Карточка теперь получает true и рисует заполненное сердце. Details для id 1 покажет Remove from favorites. Favorites.filter оставит запись 1.

Сценарий 3. Удалить последнее избранное

В Favorites у всех показанных карточек isFavorite = true: они уже прошли фильтр. Нажатие на сердце удаляет id из общего набора. favorites вычисляется заново и становится пустым. if (favorites.isEmpty()) выбирает EmptyState. Текст не спрятан поверх пустого списка: в этом случае сам LazyColumn вообще не входит в UI-описание этой ветки.

Почему нет отдельного обновления экрана

В обработчиках нет вызова «перерисовать всё». Изменяется состояние, от которого вычисляется UI. Compose связывает чтение состояния с пересчётом. Это главное объяснение того, почему одна строка присваивания меняет несколько представлений одной записи.

Ресурсы, Manifest и Gradle: разбор служебных файлов

Эти файлы редко спрашивают построчно на короткой защите, но они объясняют, почему приложение собирается, где берутся строки и как Android узнаёт точку входа. Внутри Gradle .kts тоже Kotlin, но функции plugins, android и dependencies предоставлены Gradle-плагинами, а не библиотекой UI.

XML состоит из тегов и атрибутов. <string name="buy">Buy USD</string> имеет открывающий тег, содержимое и закрывающий тег. <path ... /> - тег без дочернего содержимого. xmlns:android задаёт пространство имён для android:-атрибутов. @string/... и @drawable/... в XML - ссылки на ресурсы; в Kotlin аналогичные идентификаторы читаются через R.

AndroidManifest.xml

Путь: app/src/main/AndroidManifest.xml

Этот XML сообщает Android о приложении и его входе. Сам Compose UI здесь не описывается.

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android">
```

Строки 1-2: XML-декларация задаёт формат/кодировку; manifest с xmlns:android открывает корневой тег и пространство имён android-атрибутов.

```
4      <application
5          android:allowBackup="true"
```

Строки 4-5: application описывает приложение. allowBackup разрешает системный backup допустимых локальных данных; не сохраняет автоматически Compose remember.

```
6          android:dataExtractionRules="@xml/data_extraction_rules"
7          android:fullBackupContent="@xml/backup_rules"
```

Строки 6-7: Ссылки на новые dataExtractionRules и старые fullBackupContent backup-правила в res/xml.

```
8          android:icon="@mipmap/ic_launcher"
9          android:label="@string/app_name"
10         android:roundIcon="@mipmap/ic_launcher_round"
```

Строки 8-10: icon и roundIcon ссылаются на mipmap, label - на app_name. Это иконка/имя в launcher, а не заголовок TopAppBar.

```
11         android:supportsRtl="true"
12         android:theme="@style/Theme.RateRadar">
```

Строки 11-12: supportsRtl разрешает RTL-раскладку; theme указывает XML-стиль окна Theme.RateRadar.

```
13          <activity
14              android:name=".MainActivity"
15              android:exported="true">
```

Строки 13-15: activity объявляет MainActivity. Имя .MainActivity относительное к namespace проекта; exported=true даёт launcher возможность её открыть.

```
16              <intent-filter>
17                  <action android:name="android.intent.action.MAIN" />
```

Строки 16-17: intent-filter с действием MAIN отмечает главный вход.

```
19                  <category android:name="android.intent.category.LAUNCHER" />
20              </intent-filter>
21          </activity>
```

Строки 19-21: Категория LAUNCHER делает вход доступным из launcher. Закрываются filter и activity.

```
22      </application>
23
24  </manifest>
```

Строки 22-24: Закрываются application и manifest. Нет uses-permission INTERNET.

settings.gradle.kts

Путь: settings.gradle.kts

Настройки всего проекта: источники зависимостей и подключённые модули.

```
1  pluginManagement {
2      repositories {
```

Строки 1-2: pluginManagement и repositories задают, где Gradle ищет плагины сборки.

```
3          google()
```

Строка 3: google() - репозиторий Google, где есть Android-плагин и AndroidX.

```
4          mavenCentral()
```

Строка 4: mavenCentral() - общий репозиторий JVM/Kotlin-библиотек.

```
5          gradlePluginPortal()
6      }
7  }
```

Строки 5-7: gradlePluginPortal() - каталог Gradle-плагинов. Закрываются repositories и pluginManagement.

```
9  dependencyResolutionManagement {
10     repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
```

Строки 9-10: dependencyResolutionManagement задаёт поиск библиотек. FAIL_ON_PROJECT_REPOS запрещает отдельным модулям самовольно задавать свои repositories.

```
11      repositories {
12          google()
13          mavenCentral()
14      }
15  }
```

Строки 11-15: Библиотеки ищутся в google и mavenCentral. Закрывается блок.

```
17  rootProject.name = "RateRadarKZ"
```

Строка 17: Имя Gradle-проекта RateRadarKZ. Папка на диске всё равно может называться kbtu_android.

```
18  include(":app")
```

Строка 18: include(":app") подключает app как модуль сборки. Папка sis2 здесь не включена.

build.gradle.kts

Путь: build.gradle.kts

Это версии инструментов сборки, а не версия устанавливаемого приложения.

```
1  plugins {
```

Строка 1: plugins - блок плагинов корневого проекта.

```
2      id("com.android.application") version "8.10.1" apply false
```

Строка 2: Android application plugin 8.10.1. apply false означает объявить версию, не применять plugin к корневому модулю.

```
3      id("org.jetbrains.kotlin.android") version "2.1.21" apply false
```

Строка 3: Kotlin Android plugin 2.1.21 для компиляции Kotlin в Android-модуле, тоже apply false.

```
4      id("org.jetbrains.kotlin.plugin.compose") version "2.1.21" apply false
5  }
```

Строки 4-5: Compose compiler plugin той же Kotlin-версии 2.1.21; apply false. Закрывается plugins. Реально плагины применятся в app/build.gradle.kts.

build.gradle.kts

Путь: app/build.gradle.kts

DSL-конфигурация сборки. implementation означает подключение библиотечного кода для модуля, а не вызов функции пользовательского интерфейса.

```
1  plugins {
2      id("com.android.application")
3      id("org.jetbrains.kotlin.android")
4      id("org.jetbrains.kotlin.plugin.compose")
5  }
```

Строки 1-5: plugins применяет три ранее объявленных plugin к модулю app: Android application, Kotlin Android, Kotlin Compose.

```
7  android {
8      namespace = "kz.edu.kbtu.rateradar"
9      compileSdk = 36
```

Строки 7-9: android - конфигурация Android-модуля. namespace задаёт пространство имён ресурсов R и BuildConfig. compileSdk=36 - доступные API при компиляции.

```
11     defaultConfig {
12         applicationId = "kz.edu.kbtu.rateradar"
```

Строки 11-12: defaultConfig - общие настройки варианта. applicationId - уникальное имя устанавливаемого пакета.

```
13         minSdk = 24
14         targetSdk = 36
```

Строки 13-14: minSdk 24 - нижний предел установки; targetSdk 36 - заявленный уровень ожидаемого Android-поведения.

```
15         versionCode = 1
16         versionName = "1.0"
```

Строки 15-16: versionCode=1 - целочисленный номер версии для обновлений; versionName="1.0" - отображаемая человеку версия.

```
18         testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
19         vectorDrawables.useSupportLibrary = true
20     }
```

Строки 18-20: testInstrumentationRunner - runner Android-тестов. vectorDrawables.useSupportLibrary - настройка поддержки vector resources. Закрывается defaultConfig.

```
22     buildTypes {
23         release {
24             isMinifyEnabled = false
```

Строки 22-24: buildTypes/release: isMinifyEnabled=false отключает уменьшение и обфускацию кода R8 для release.

```
25              proguardFiles(
26                  getDefaultProguardFile("proguard-android-optimize.txt"),
27                  "proguard-rules.pro",
28              )
29          }
30      }
```

Строки 25-30: proguardFiles перечисляет стандартные и проектные правила R8. При выключенной minification они не выполняют полноценную оптимизацию. Закрываются вызов, release и buildTypes.

```
32      compileOptions {
33          sourceCompatibility = JavaVersion.VERSION_17
34          targetCompatibility = JavaVersion.VERSION_17
35      }
```

Строки 32-35: compileOptions задаёт Java source/target compatibility 17. Это не версия Android и не требование использовать именно JDK 17 для любого запуска Gradle.

```
37      kotlinOptions {
38          jvmTarget = "17"
39      }
```

Строки 37-39: kotlinOptions.jvmTarget="17" - целевой JVM bytecode Kotlin. Согласован с Java target.

```
41      buildFeatures {
42          compose = true
43          buildConfig = true
44      }
```

Строки 41-44: buildFeatures включает compose и создание BuildConfig.

```
46      packaging {
47          resources.excludes += "/META-INF/{AL2.0,LGPL2.1}"
48      }
49  }
```

Строки 46-49: packaging исключает совпадающие META-INF лицензионные файлы AL2.0/LGPL2.1 при упаковке; фигурные скобки в строке - шаблон имён. Закрывается android.

```
51  dependencies {
52      val composeBom = platform("androidx.compose:compose-bom:2025.06.01")
```

Строки 51-52: dependencies перечисляет библиотеки. platform(compose-bom) импортирует согласованные версии Compose; BOM не содержит сами компоненты UI.

```
54      implementation(composeBom)
55      androidTestImplementation(composeBom)
```

Строки 54-55: BOM применяется и к обычным зависимостям, и к androidTest. Для большинства Compose-зависимостей версию поэтому отдельно не пишут.

```
57      implementation("androidx.core:core-ktx:1.16.0")
```

Строка 57: core-ktx - Kotlin-удобства AndroidX для базовых API.

```
58      implementation("androidx.activity:activity-compose:1.10.1")
```

Строка 58: activity-compose - связка Activity с Compose, в том числе setContent.

```
59      implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.9.1")
```

Строка 59: lifecycle-runtime-ktx - Kotlin-поддержка lifecycle. Прямого собственного lifecycle-кода здесь нет.

```
60      implementation("androidx.compose.ui:ui")
```

Строка 60: compose.ui:ui - базовые UI-типы, Modifier, элементы инфраструктуры.

```
61      implementation("androidx.compose.ui:ui-graphics")
```

Строка 61: ui-graphics - типы рисования, Color, ImageVector и другое.

```
62      implementation("androidx.compose.ui:ui-tooling-preview")
```

Строка 62: ui-tooling-preview - API аннотаций и параметров Preview.

```
63      implementation("androidx.compose.material3:material3")
```

Строка 63: material3 - Scaffold, TopAppBar, Text, Button и оформление Material 3.

```
64      implementation("androidx.compose.material:material-icons-core")
```

Строка 64: material-icons-core - использованный базовый набор иконок.

```
65      implementation("androidx.navigation:navigation-compose:2.9.0")
```

Строка 65: navigation-compose 2.9.0 - NavHost, composable, NavController для Compose.

```
67      debugImplementation("androidx.compose.ui:ui-tooling")
68      debugImplementation("androidx.compose.ui:ui-test-manifest")
```

Строки 67-68: debugImplementation - только debug: инструменты Preview и manifest поддержки UI-тестов.

```
70      testImplementation("junit:junit:4.13.2")
```

Строка 70: JUnit для локальных тестов в src/test.

```
71      androidTestImplementation("androidx.test.ext:junit:1.2.1")
72      androidTestImplementation("androidx.test.espresso:espresso-core:3.6.1")
73      androidTestImplementation("androidx.compose.ui:ui-test-junit4")
```

Строки 71-73: androidTestImplementation - инструменты тестов на Android: JUnit runner, Espresso, Compose UI tests. Они подключены, но написанных androidTest-файлов сейчас нет.

```
74  }
```

Строка 74: Закрывается dependencies. Наличие зависимости не создаёт фичу автоматически.

gradle.properties

Путь: gradle.properties

Ключ=значение, это не Kotlin-файл.

```
1  org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
```

Строка 1: org.gradle.jvmargs: максимум heap Gradle 2048 MB и UTF-8 для текстовой кодировки.

```
2  org.gradle.parallel=true
```

Строка 2: parallel=true разрешает параллельную работу Gradle там, где это возможно.

```
3  org.gradle.caching=true
```

Строка 3: caching=true включает использование build cache.

```
4  android.useAndroidX=true
```

Строка 4: useAndroidX=true выбирает семейство AndroidX библиотек.

```
5  kotlin.code.style=official
```

Строка 5: kotlin.code.style=official выбирает стандартный стиль Kotlin.

```
6  android.nonTransitiveRClass=true
```

Строка 6: nonTransitiveRClass=true ограничивает R модуля его собственными ресурсами вместо повторения ресурсов зависимостей.

gradle-wrapper.properties

Путь: gradle/wrapper/gradle-wrapper.properties

Gradle wrapper закрепляет инструмент сборки. Его версия отличается от версии Android Gradle Plugin.

```
1  distributionBase=GRADLE_USER_HOME
2  distributionPath=wrapper/dists
```

Строки 1-2: База GRADLE_USER_HOME и подпапка wrapper/dists указывают, куда хранить загруженную Gradle distribution.

```
3  distributionUrl=https\://services.gradle.org/distributions/gradle-8.11.1-bin.zip
```

Строка 3: distributionUrl выбирает Gradle 8.11.1, bin-архив. Экранирование https\: нужно формату .properties; это обычный URL после чтения.

```
4  networkTimeout=10000
```

Строка 4: networkTimeout=10000 - timeout скачивания distribution в миллисекундах.

```
5  validateDistributionUrl=true
```

Строка 5: validateDistributionUrl=true включает проверку URL distribution.

```
6  zipStoreBase=GRADLE_USER_HOME
7  zipStorePath=wrapper/dists
```

Строки 6-7: zipStoreBase/zipStorePath задают хранение zip-архива. gradlew использует эти значения через wrapper.jar.

ExampleUnitTest.kt

Путь: app/src/test/java/kz/edu/kbtu/rateradar/ExampleUnitTest.kt

Локальный тест запускается на JVM компьютера, а не на экране эмулятора.

```
1  package kz.edu.kbtu.rateradar
2
3  import org.junit.Assert.assertEquals
4  import org.junit.Test
```

Строка 1: package задаёт адрес объявлений: kz.edu.kbtu.rateradar. Это пространство имён, не код запуска.

Строка 3: assertEquals. JUnit-проверка равенства expected и actual.

Строка 4: Test. Аннотация метода unit-теста.

```
6  class ExampleUnitTest {
```

Строка 6: Класс группы JUnit-тестов.

```
7      @Test
8      fun additionIsCorrect() {
```

Строки 7-8: @Test отмечает метод, который запускает testDebugUnitTest.

```
9          assertEquals(4, 2 + 2)
```

Строка 9: assertEquals ожидает 4 и сравнивает с результатом 2+2. Не проверяет навигацию, фильтр, state или UI.

```
10     }
11 }
```

Строки 10-11: Закрываются метод и класс. Это шаблонный тест, не свидетельство полной корректности приложения.

strings.xml

Путь: app/src/main/res/values/strings.xml

Строки позволяют менять надписи без поиска каждого Text. Имена/города/адреса демоданных остаются в SampleData, Preview-образцы могут содержать литералы.

```
1  <resources>
2      <string name="app_name">RateRadar KZ</string>
```

Строки 1-2: resources - корень; app_name - имя приложения для Manifest.

```
4       <!-- Screen titles -->
5       <string name="title_offices">Exchange Offices</string>
6       <string name="title_details">Office Details</string>
7       <string name="title_favorites">Favorites</string>
```

Строки 4-7: Комментарий Screen titles и три ресурса заголовков. name - постоянный id, содержимое - отображаемый текст.

```
9       <!-- Buttons and icons -->
10      <string name="open_favorites">Open favorites</string>
11      <string name="back">Back</string>
12      <string name="add_to_favorites">Add to favorites</string>
13      <string name="remove_from_favorites">Remove from favorites</string>
```

Строки 9-13: Тексты описаний/действий кнопок. Add/Remove используются и у сердца, и у Details button; back описывает стрелку.

```
15      <!-- Rates -->
16      <string name="buy">Buy USD</string>
17      <string name="sell">Sell USD</string>
```

Строки 15-17: Комментарий Rates; buy и sell задают подписи Buy USD / Sell USD.

```
18      <string name="rate_value">%1$.2f ☐</string>
```

Строка 18: Формат %1$.2f: первый аргумент, floating-point, два знака после десятичного разделителя; далее символ тенге. stringResource передаёт rate Double. Разделитель может зависеть от locale.

```
19      <string name="city_and_time">%1$s · updated %2$s</string>
```

Строка 19: Формат %1$s - первый строковый аргумент city, %2$s - второй updatedAt. Символ · разделяет город и время.

```
21      <!-- Details -->
22      <string name="office_image_description">Photo of the %1$s exchange office</string>
```

Строки 21-22: Описание изображения подставляет office.name. Оно должно обозначать иллюстрацию, а не реальное фото.

```
23      <string name="label_city">City: %1$s</string>
24      <string name="label_address">Address: %1$s</string>
25      <string name="label_hours">Working hours: %1$s</string>
```

Строки 23-25: City, Address, Working hours получают первый строковый аргумент через %1$s.

```
27      <!-- Empty states -->
28      <string name="empty_favorites_title">No favorites yet</string>
29      <string name="empty_favorites_message">Tap the heart on an office to save it
    here.</string>
```

Строки 27-29: Тексты пустого избранного: заголовок и инструкция добавить сердце.

```
30      <string name="empty_city_title">No offices here</string>
31      <string name="empty_city_message">There are no exchange offices in this city yet.
    Try another city.</string>
```

Строки 30-31: Тексты пустого результата фильтра: попробуй другой город.

```
32      <string name="office_not_found_title">Office not found</string>
33      <string name="office_not_found_message">This exchange office does not
    exist.</string>
34  </resources>
```

Строки 32-34: Тексты неизвестного обменника, затем закрывается resources.

themes.xml

Путь: app/src/main/res/values/themes.xml

XML-тема окна, отдельно от ui/theme/Theme.kt. Android автоматически выбирает values-night по ночной конфигурации.

```
1   <resources>
2       <style name="Theme.RateRadar"
    parent="android:style/Theme.Material.Light.NoActionBar">
```

Строки 1-2: resources и style Theme.RateRadar. Родитель системной Material светлой темы без ActionBar, чтобы стандартная панель не дублировала Compose TopAppBar.

```
3           <item name="android:fontFamily">sans</item>
```

Строка 3: android:fontFamily=sans выбирает системное sans-семейство для окна/традиционных Android-виджетов. Это не кастомный Compose-шрифт.

```
4           <item name="android:windowLightStatusBar">true</item>
```

Строка 4: windowLightStatusBar=true задаёт, нужны ли тёмные значки status bar. Название про светлый фон, а не «белые значки».

```
5           <item name="android:statusBarColor">@android:color/transparent</item>
```

Строка 5: statusBarColor - прозрачный системный цвет. При edge-to-edge фон за панелью задаёт окно/контент.

```
6           <item name="android:navigationBarColor">#FAFCFF</item>
```

Строка 6: navigationBarColor - XML-цвет системной нижней панели; его применение может зависеть от Android и режима навигации.

```
7           <item name="android:windowActionModeOverlay">true</item>
```

Строка 7: windowActionModeOverlay позволяет contextual action mode лежать поверх содержимого.

```
8          <item name="android:windowNoTitle">true</item>
9      </style>
10 </resources>
```

Строки 8-10: windowNoTitle убирает системный заголовок окна; закрываются style и resources.

themes.xml

Путь: app/src/main/res/values-night/themes.xml

XML-тема окна, отдельно от ui/theme/Theme.kt. Android автоматически выбирает values-night по ночной конфигурации.

```
1  <resources>
2      <style name="Theme.RateRadar" parent="android:style/Theme.Material.NoActionBar">
```

Строки 1-2: resources и style Theme.RateRadar. Родитель системной Material тёмной темы без ActionBar, чтобы стандартная панель не дублировала Compose TopAppBar.

```
3          <item name="android:fontFamily">sans</item>
```

Строка 3: android:fontFamily=sans выбирает системное sans-семейство для окна/традиционных Android-виджетов. Это не кастомный Compose-шрифт.

```
4          <item name="android:windowLightStatusBar">false</item>
```

Строка 4: windowLightStatusBar=false задаёт, нужны ли тёмные значки status bar. Название про светлый фон, а не «белые значки».

```
5          <item name="android:statusBarColor">@android:color/transparent</item>
```

Строка 5: statusBarColor - прозрачный системный цвет. При edge-to-edge фон за панелью задаёт окно/контент.

```
6          <item name="android:navigationBarColor">#0B1520</item>
```

Строка 6: navigationBarColor - XML-цвет системной нижней панели; его применение может зависеть от Android и режима навигации.

```
7          <item name="android:windowActionModeOverlay">true</item>
```

Строка 7: windowActionModeOverlay позволяет contextual action mode лежать поверх содержимого.

```
8          <item name="android:windowNoTitle">true</item>
9      </style>
10 </resources>
```

Строки 8-10: windowNoTitle убирает системный заголовок окна; закрываются style и resources.

colors.xml

Путь: app/src/main/res/values/colors.xml

Цвет launcher-ресурса.

```
<resources>
    <color name="launcher_background">#0A1F33</color>
</resources>
```

Строки 1-3: Единственный ресурс launcher_background для иконки приложения, затем закрывается resources. Он не используется как цвет карточек Compose.

exchange_placeholder.xml

Путь: app/src/main/res/drawable/exchange_placeholder.xml

Это векторная иллюстрация. Lint рекомендует не делать некоторые vector resources больше 200x200 dp; это warning, не ошибка сборки.

```
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="360dp"
    android:height="180dp"
    android:viewportWidth="360"
    android:viewportHeight="180">
```

Строки 1-5: vector: пространство имён, размер 360x180 dp и внутренняя координатная область 360x180. viewport - единицы рисунка, не пиксели дисплея.

```
    <!-- background -->
    <path
        android:fillColor="#CCE8E2"
        android:pathData="M0,0h360v180h-360z" />
```

Строки 6-9: Фон: path с fillColor и pathData. M0,0 переносит начало, h360 идёт вправо, v180 вниз, h-360 назад, z замыкает прямоугольник.

```
    <!-- ground -->
    <path
        android:fillColor="#9EC9C1"
        android:pathData="M0,150h360v30h-360z" />
```

Строки 10-13: Ground: нижний прямоугольник от y=150 высотой 30.

```
    <!-- building -->
    <path
        android:fillColor="#FFFFFF"
        android:pathData="M110,60h140v90h-140z" />
```

Строки 14-17: Building: белый прямоугольник здания, M110,60, ширина 140, высота 90.

```
    <!-- roof -->
    <path
        android:fillColor="#006A60"
        android:pathData="M100,45h160v18h-160z" />
```

Строки 18-21: Roof: прямоугольная крыша с fillColor #006A60.

```
22      <!-- sign -->
23      <path
24          android:fillColor="#00201C"
25          android:pathData="M140,70h80v20h-80z" />
```

Строки 22-25: Sign: тёмный прямоугольник вывески на здании.

```
26      <!-- door -->
27      <path
28          android:fillColor="#4A635F"
29          android:pathData="M165,105h30v45h-30z" />
```

Строки 26-29: Door: прямоугольник двери.

```
30      <!-- windows -->
31      <path
32          android:fillColor="#82D5C8"
33          android:pathData="M122,105h30v25h-30z M208,105h30v25h-30z" />
```

Строки 30-33: Windows: один path содержит два замкнутых прямоугольника окон; второй M начинает новый контур.

```
34      <!-- coin -->
35      <path
36          android:fillColor="#F2C14E"
37          android:pathData="M300,60m-24,0a24,24 0,1 1,48 0a24,24 0,1 1,-48 0" />
```

Строки 34-37: Coin: M задаёт центр, m смещает начало, a задаёт дуги окружности; два дуговых сегмента образуют круг радиуса 24.

```
38      <path
39          android:fillColor="#00201C"
40          android:pathData="M290,48h20v4h-20z M290,56h20v4h-20z M298,60h4v16h-4z" />
41  </vector>
```

Строки 38-41: Последний path рисует знак из полос на монете; закрывается vector. Цвета - часть иллюстрации и не автоматически читают primary темы.

ic_launcher_foreground.xml

Путь: app/src/main/res/drawable/ic_launcher_foreground.xml

Иконка приложения для launcher. anydpi - ресурс, подходящий разным плотностям; v26 - вариант для API 26 и выше. Обычная и round версии поддерживают формы разных launcher.

```
1  <?xml version="1.0" encoding="utf-8"?>
```

Строка 1: XML-декларация, UTF-8.

```
2  <vector xmlns:android="http://schemas.android.com/apk/res/android"
```

Строка 2: Открывается векторный рисунок с Android-атрибутами.

```
3       android:width="108dp"
```

Строка 3: Внешняя ширина drawable в dp.

```
4       android:height="108dp"
```

Строка 4: Внешняя высота drawable в dp.

```
5       android:viewportWidth="108"
```

Строка 5: Ширина внутренней координатной области рисунка.

```
6       android:viewportHeight="108">
```

Строка 6: Высота внутренней координатной области; завершается открывающий тег vector.

```
7       <path
```

Строка 7: Начинается отдельный нарисованный контур.

```
8           android:fillColor="#2DD4BF"
```

Строка 8: Цвет заливки этого контура, часть launcher-рисунка.

```
9           android:pathData="M54,18A36,36 0,1 0,54 90A36,36 0,1 0,54 18" />
```

Строка 9: Геометрия контура: M перенос, h/v прямые, A дуги, l относительная линия, z замыкание. Завершается path.

```
10      <path
```

Строка 10: Начинается отдельный нарисованный контур.

```
11          android:fillColor="#0A1F33"
```

Строка 11: Цвет заливки этого контура, часть launcher-рисунка.

```
12          android:pathData="M35,38h38v7h-38zM35,51h25v7h-25zM35,64h38v7h-38z" />
```

Строка 12: Геометрия контура: M перенос, h/v прямые, A дуги, l относительная линия, z замыкание. Завершается path.

```
13      <path
```

Строка 13: Начинается отдельный нарисованный контур.

```
14          android:fillColor="#FFFFFF"
```

Строка 14: Цвет заливки этого контура, часть launcher-рисунка.

```
15          android:pathData="M66,47l10,7 -10,7z" />
```

Строка 15: Геометрия контура: M перенос, h/v прямые, A дуги, l относительная линия, z замыкание. Завершается path.

```
16  </vector>
```

Строка 16: Закрывается вектор.

ic_launcher.xml

Путь: app/src/main/res/mipmap-anydpi/ic_launcher.xml

Иконка приложения для launcher. anydpi - ресурс, подходящий разным плотностям; v26 - вариант для API 26 и выше. Обычная и round версии поддерживают формы разных launcher.

```
1  <?xml version="1.0" encoding="utf-8"?>
```

Строка 1: XML-декларация, UTF-8.

```
2  <vector xmlns:android="http://schemas.android.com/apk/res/android"
```

Строка 2: Открывается векторный рисунок с Android-атрибутами.

```
3      android:width="48dp"
```

Строка 3: Внешняя ширина drawable в dp.

```
4      android:height="48dp"
```

Строка 4: Внешняя высота drawable в dp.

```
5      android:viewportWidth="108"
```

Строка 5: Ширина внутренней координатной области рисунка.

```
6      android:viewportHeight="108">
```

Строка 6: Высота внутренней координатной области; завершается открывающий тег vector.

```
7      <path
```

Строка 7: Начинается отдельный нарисованный контур.

```
8          android:fillColor="#0A1F33"
```

Строка 8: Цвет заливки этого контура, часть launcher-рисунка.

```
9          android:pathData="M0,0h108v108h-108z" />
```

Строка 9: Геометрия контура: M перенос, h/v прямые, A дуги, l относительная линия, z замыкание. Завершается path.

```
10      <path
```

Строка 10: Начинается отдельный нарисованный контур.

```
11          android:fillColor="#2DD4BF"
```

Строка 11: Цвет заливки этого контура, часть launcher-рисунка.

```
12          android:pathData="M54,18A36,36 0,1 0,54 90A36,36 0,1 0,54 18" />
```

Строка 12: Геометрия контура: M перенос, h/v прямые, A дуги, l относительная линия, z замыкание. Завершается path.

```
13      <path
```

Строка 13: Начинается отдельный нарисованный контур.

```
14          android:fillColor="#0A1F33"
```

Строка 14: Цвет заливки этого контура, часть launcher-рисунка.

```
15          android:pathData="M35,38h38v7h-38zM35,51h25v7h-25zM35,64h38v7h-38z" />
```

Строка 15: Геометрия контура: M перенос, h/v прямые, A дуги, l относительная линия, z замыкание. Завершается path.

```
16      <path
```

Строка 16: Начинается отдельный нарисованный контур.

```
17          android:fillColor="#FFFFFF"
```

Строка 17: Цвет заливки этого контура, часть launcher-рисунка.

```
18          android:pathData="M66,47l10,7 -10,7z" />
```

Строка 18: Геометрия контура: M перенос, h/v прямые, A дуги, l относительная линия, z замыкание. Завершается path.

```
19  </vector>
```

Строка 19: Закрывается вектор.

ic_launcher_round.xml

Путь: app/src/main/res/mipmap-anydpi/ic_launcher_round.xml

Иконка приложения для launcher. anydpi - ресурс, подходящий разным плотностям; v26 - вариант для API 26 и выше. Обычная и round версии поддерживают формы разных launcher.

```
1  <?xml version="1.0" encoding="utf-8"?>
```

Строка 1: XML-декларация, UTF-8.

```
2  <vector xmlns:android="http://schemas.android.com/apk/res/android"
```

Строка 2: Открывается векторный рисунок с Android-атрибутами.

```
3      android:width="48dp"
```

Строка 3: Внешняя ширина drawable в dp.

```
4      android:height="48dp"
```

Строка 4: Внешняя высота drawable в dp.

```
5      android:viewportWidth="108"
```

Строка 5: Ширина внутренней координатной области рисунка.

```
6      android:viewportHeight="108">
```

Строка 6: Высота внутренней координатной области; завершается открывающий тег vector.

```
7      <path
```

Строка 7: Начинается отдельный нарисованный контур.

```
8          android:fillColor="#0A1F33"
```

Строка 8: Цвет заливки этого контура, часть launcher-рисунка.

```
9          android:pathData="M54,0A54,54 0,1 0,54 108A54,54 0,1 0,54 0" />
```

Строка 9: Геометрия контура: M перенос, h/v прямые, A дуги, l относительная линия, z замыкание. Завершается path.

```
10     <path
```

Строка 10: Начинается отдельный нарисованный контур.

```
11         android:fillColor="#2DD4BF"
```

Строка 11: Цвет заливки этого контура, часть launcher-рисунка.

```
12         android:pathData="M54,18A36,36 0,1 0,54 90A36,36 0,1 0,54 18" />
```

Строка 12: Геометрия контура: M перенос, h/v прямые, A дуги, l относительная линия, z замыкание. Завершается path.

```
13      <path
```

Строка 13: Начинается отдельный нарисованный контур.

```
14          android:fillColor="#0A1F33"
```

Строка 14: Цвет заливки этого контура, часть launcher-рисунка.

```
15          android:pathData="M35,38h38v7h-38zM35,51h25v7h-25zM35,64h38v7h-38z" />
```

Строка 15: Геометрия контура: M перенос, h/v прямые, A дуги, l относительная линия, z замыкание. Завершается path.

```
16      <path
```

Строка 16: Начинается отдельный нарисованный контур.

```
17          android:fillColor="#FFFFFF"
```

Строка 17: Цвет заливки этого контура, часть launcher-рисунка.

```
18          android:pathData="M66,47l10,7 -10,7z" />
```

Строка 18: Геометрия контура: M перенос, h/v прямые, A дуги, l относительная линия, z замыкание. Завершается path.

```
19  </vector>
```

Строка 19: Закрывается вектор.

ic_launcher.xml

Путь: app/src/main/res/mipmap-anydpi-v26/ic_launcher.xml

Иконка приложения для launcher. anydpi - ресурс, подходящий разным плотностям; v26 - вариант для API 26 и выше. Обычная и round версии поддерживают формы разных launcher.

```
1  <?xml version="1.0" encoding="utf-8"?>
```

Строка 1: XML-декларация, UTF-8.

```
2  <adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
```

Строка 2: Адаптивная launcher-иконка Android 8/API 26+: система накладывает форму на слои.

```
3      <background android:drawable="@color/launcher_background" />
```

Строка 3: Фоновый слой ссылается на launcher_background из colors.xml.

```
4      <foreground android:drawable="@drawable/ic_launcher_foreground" />
```

Строка 4: Передний слой ссылается на vector ic_launcher_foreground.

```
5  </adaptive-icon>
```

Строка 5: Закрывается adaptive-icon. Monochrome-слой не задан; Lint выдаёт warning для тематических иконок.

ic_launcher_round.xml

Путь: app/src/main/res/mipmap-anydpi-v26/ic_launcher_round.xml

Иконка приложения для launcher. anydpi - ресурс, подходящий разным плотностям; v26 - вариант для API 26 и выше. Обычная и round версии поддерживают формы разных launcher.

```
1  <?xml version="1.0" encoding="utf-8"?>
```

Строка 1: XML-декларация, UTF-8.

```
2  <adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
```

Строка 2: Адаптивная launcher-иконка Android 8/API 26+: система накладывает форму на слои.

```
3      <background android:drawable="@color/launcher_background" />
```

Строка 3: Фоновый слой ссылается на launcher_background из colors.xml.

```
4      <foreground android:drawable="@drawable/ic_launcher_foreground" />
```

Строка 4: Передний слой ссылается на vector ic_launcher_foreground.

```
5  </adaptive-icon>
```

Строка 5: Закрывается adaptive-icon. Monochrome-слой не задан; Lint выдаёт warning для тематических иконок.

backup_rules.xml

Путь: app/src/main/res/xml/backup_rules.xml

Шаблонные правила системного переноса данных. Они не записывают favoriteIds из RAM на диск.

```
1  <?xml version="1.0" encoding="utf-8"?>
```

Строка 1: XML-декларация.

```
2  <full-backup-content>
```

Строка 2: Открывает старый формат правил системного backup.

```
3      <exclude domain="sharedpref" path="device.xml" />
```

Строка 3: Не копировать shared preference файл device.xml. Такого своего хранилища в текущем коде нет.

```
4  </full-backup-content>
```

Строка 4: Закрывает соответствующий XML-блок.

data_extraction_rules.xml

Путь: app/src/main/res/xml/data_extraction_rules.xml

Шаблонные правила системного переноса данных. Они не записывают favoriteIds из RAM на диск.

```
1  <?xml version="1.0" encoding="utf-8"?>
```

Строка 1: XML-декларация.

```
2  <data-extraction-rules>
```

Строка 2: Открывает новый формат extraction rules.

```
3      <cloud-backup>
```

Строка 3: Правила облачного резервного копирования.

```
4          <exclude domain="sharedpref" path="device.xml" />
```

Строка 4: Не копировать shared preference файл device.xml. Такого своего хранилища в текущем коде нет.

```
5      </cloud-backup>
```

Строка 5: Закрывает соответствующий XML-блок.

```
6      <device-transfer>
```

Строка 6: Правила переноса на другое устройство.

```
7          <exclude domain="sharedpref" path="device.xml" />
```

Строка 7: Не копировать shared preference файл device.xml. Такого своего хранилища в текущем коде нет.

```
8      </device-transfer>
```

Строка 8: Закрывает соответствующий XML-блок.

```
9  </data-extraction-rules>
```

Строка 9: Закрывает соответствующий XML-блок.

proguard-rules.pro

Путь: app/proguard-rules.pro

Файл подключён в release-конфигурации; minify выключен.

```
1  # Project-specific R8 rules can be added here when a release build needs them.
```

Строка 1: Комментарий: сюда при необходимости добавляют проектные R8-правила. Сейчас собственных правил нет.

.gitignore

Путь: .gitignore

Правила, какие локальные файлы Git не должен добавлять. Уже отслеживаемый файл не перестаёт отслеживаться только из-за нового правила.

```
1  .gradle/
```

Строка 1: Игнорировать локальную служебную папку или файл .gradle/. Правило Git не удаляет файл с диска.

```
2  .kotlin/
```

Строка 2: Игнорировать локальную служебную папку или файл .kotlin/. Правило Git не удаляет файл с диска.

```
3  .idea/
```

Строка 3: Игнорировать локальную служебную папку или файл .idea/. Правило Git не удаляет файл с диска.

```
4  .firecrawl/
```

Строка 4: Игнорировать локальную служебную папку или файл .firecrawl/. Правило Git не удаляет файл с диска.

```
5  tmp/
```

Строка 5: Игнорировать локальную служебную папку или файл tmp/. Правило Git не удаляет файл с диска.

```
6  local.properties
```

Строка 6: Локальная конфигурация SDK не должна уходить в репозиторий.

```
7  *.iml
```

Строка 7: Игнорировать локальную служебную папку или файл *.iml. Правило Git не удаляет файл с диска.

```
8  .DS_Store
```

Строка 8: Игнорировать локальную служебную папку или файл .DS_Store. Правило Git не удаляет файл с диска.

```
10  # Generated build output
```

Строка 10: Комментарий описывает следующую группу игнорируемых файлов.

```
11  build/
```

Строка 11: Игнорировать сгенерированный результат сборки, включая вложенные build.

```
12  **/build/
```

Строка 12: Игнорировать сгенерированный результат сборки, включая вложенные build.

```
13  captures/
```

Строка 13: Игнорировать локальную служебную папку или файл captures/. Правило Git не удаляет файл с диска.

```
14  .externalNativeBuild/
```

Строка 14: Игнорировать локальную служебную папку или файл .externalNativeBuild/. Правило Git не удаляет файл с диска.

```
15  .cxx/
```

Строка 15: Игнорировать локальную служебную папку или файл .cxx/. Правило Git не удаляет файл с диска.

```
17  # Local signing material
```

Строка 17: Комментарий описывает следующую группу игнорируемых файлов.

```
18  *.jks
```

Строка 18: Не коммитить локальные ключи подписи.

```
19  *.keystore
```

Строка 19: Не коммитить локальные ключи подписи.

```
20  sis-files/
```

Строка 20: Локальная папка исходных материалов заданий; не часть source set.

gradlew, gradlew.bat, wrapper.jar и локальные файлы

gradlew - сгенерированный shell-скрипт запуска Gradle на macOS/Linux: определяет Java и запускает GradleWrapperMain из wrapper.jar. gradlew.bat - Windows-вариант. wrapper.jar - бинарная реализация wrapper. Не нужно менять их для карточки или фильтра. ./gradlew означает выполнить скрипт из текущей папки; assembleDebug - имя задачи сборки.

local.properties хранит путь sdk.dir на твоём компьютере. В книге его полный текст не копируется: это локальная конфигурация. .idea - настройки Android Studio; .gradle/.kotlin - кэши. build/intermediates -

промежуточные результаты, build/outputs - APK/отчёты. В них не редактируют исходный UI.

README, design, screenshots, AI_USAGE, docs

README.md - описание проекта для преподавателя и других читателей. Screenshots ссылаются на существующие PNG, design/README объясняет три эскиза. AI_USAGE.md должен описывать реально использованные инструменты и конкретный случай ошибки. docs/SIS1_Project_Description.md и RateRadar_KZ_SIS1.pdf сохраняют идею будущего приложения. Никакая строка Markdown не исполняется как Android UI.

Аудит SIS3: что выполнено и что нельзя назвать идеальным

Источник критериев - переданные SIS3.pdf, страницы 2-5. Источник фактов - текущие файлы, локальная история Git, анонимный GitHub API, свежая сборка/Lint и запуск на отдельном Pixel 9 emulator-5556. Это аудит, а не подтверждение преподавателя. Формальные риски указаны отдельно от работающей реализации.

Требование	Результат и доказательство
3+ экрана	Есть List, Details, Favorites; три destination в NavGraph.
LazyColumn с 10+ записями	Есть 12 обменников при All. Фильтр вправе показывать меньше.
LazyRow	Есть пять городских FilterChip.
Scaffold + TopAppBar на каждом	Есть во всех трёх функциях экранов.
Back на второстепенных экранах	Callback popBackStack; переходы проверены на эмуляторе.
Длинный текст + ellipsis	В OfficeCard название и подстрока ограничены одной строкой. В Details длинное название переносится полностью; при буквальном требовании для всех заголовков - формальный риск.
Пустой список	Есть EmptyState для Favorites и Karaganda; есть также неизвестный id.
Image из ресурсов	Есть exchange_placeholder.xml и contentDescription. Текст описания говорит Photo, хотя ресурс - векторная иллюстрация; описание стоит сделать точнее.
Своя палитра	Есть две схемы в Theme.kt, значения в Color.kt. Не все Material-роли вручную переопределены.
Нет цветов/размеров шрифта в screens	В коде экранов цветовые роли и Typography; литералов цвета/fontSize нет. Цвета в drawable являются частью рисунка; буквальную фразу «только в theme» стоит уточнить, а не объявлять автоматической ошибкой.
Единые отступы	Spacing = 4/8/16/24 dp; Sizes отдельно для размеров.
Dark mode	Есть системный выбор схемы, previews и screenshot; визуальная проверка сделана.
48 dp для кнопок/иконок	IconButton имеет стандартную Material-зону 48 dp; details button минимум Sizes.minTouchTarget. Визуальная высота FilterChip может быть 32 dp, touch target расширяется Material до 48 dp.
3 переиспользуемых компонента	OfficeCard - List/Favorites; RateBlock - OfficeCard/Details; EmptyState - все три случая.
Previews	Все три компонента и все три экрана имеют светлый и тёмный Preview.

Требование	Результат и доказательство
Data class + list	ExchangeOffice + SampleData.offices.
Navigation Compose	Подключена 2.9.0; NavHost соединяет три экрана.
id аргументом и поиск	details/{officeId}, NavType.IntType, getInt, find по id.
remember + mutableStateOf	Общее избранное; фильтр дополнительно rememberSaveable.
Папка design и подписанные эскизы	Есть три PNG с разметкой структуры.
Дизайн до кода SIS3	f0a3c06 предшествует новым screens/components. В SIS1 уже существовал собственный WelcomeScreen в MainActivity. Новые три экрана SIS3 появились после эскизов, но буквально «до любого кода экранов» и «первый коммит repo» не соблюдены; преподаватель может читать пункт в контексте нового этапа SIS3.
Эскиз на бумаге или в Figma	В AI_USAGE написано, что эскизы генерировал Claude Code. Из PNG нельзя доказать бумагу/Figma; это спорный формат и авторство шага «design first».
5+ содержательных коммитов по времени	Число есть: 7 SIS3-коммитов. Распределения по неделе нет: все 6 октября 15:38-16:14 по Алматы. Не выполнено.
README: имя, screenshots, сравнение	Есть все три экрана light/dark, empty states, описание трёх отличий. Эскиз Box в пустом Favorites и реальная Column без Box отдельно не объяснены.
Public GitHub	Анонимный API подтвердил private=false для xlmtvv/rateradar-kz. HEAD remote a9d6b88 совпадает с локальным commit.
AI_USAGE честный	Все четыре требуемых темы записаны. Копирование именно реальных prompts и ручное авторство по файлу не доказать. Фраза reviewed every file требует соответствия твоей реальной работе.
Понимать и менять код	До твоего самостоятельного объяснения и упражнения это нельзя отметить выполненным.
Ссылка отправлена в Teams	Из локального проекта проверить невозможно.

Основной установленный пробел: Git

В истории SIS3: f0a3c06 15:38:40 - эскизы; 5568198 15:40:46 - тема; 3f5ab7a 15:41:00 - данные; 2ff1b29 15:41:45 - компоненты; 015f45e 15:42:51 - экраны; 4d2d35e 16:13:04 - навигация; a9d6b88 16:14:12 - документация. Всё 06.10.2026. Более ранние коммиты SIS1 08.09 и SIS2 15.09 не доказывают распределённую по времени разработку SIS3.

SIS3 сам противоречив: в разделе 6 просит коммиты «spread over the week», в финальном checklist - «over two weeks». Здесь не выполнен даже более мягкий вариант. Новые честные коммиты могут записать дальнейшую работу, но не исправят уже состоявшиеся даты. Не подменяй даты и не дроби задним числом историю для видимости соблюдения.

Небольшие формальные вопросы

1. На эскизе Favorites пустое сообщение помещено в Box; в коде родитель - Scaffold content, EmptyState сам содержит Column. Вид похож, структура другая. Разрешено объяснить это в README: отдельный Box оказался не нужен. 2. Все названия в карточках обрезаются, но имя в Details не имеет maxLines/overflow. Это удобно для полного названия, однако для требования «точь-в-точь» можно ограничить его двумя строками с Ellipsis. 3. «Photo of ...» - неточное описание векторного placeholder: «Illustration for ... exchange office» честнее.

4. Для подтверждения design first надо честно описать, что выбирал ты, что генерировал помощник, и откуда появились PNG. Экспорт PNG из Figma сам по себе допустим, но факт Figma здесь не установлен. 5. В AI_USAGE утверждение «I reviewed every file» не стоит оставлять как совершённый факт, если сейчас ты только начинаешь разбираться. После реального изучения и самостоятельных правок можно описать конкретные выполненные действия.

Есть ли лишнее

В SIS3 нет запрета на любые дополнительные строки кода. Ненужных больших подсистем нет: нет Room, Retrofit, Hilt, ViewModel, сетевого клиента, авторизации и бонусной bottom navigation. Karaganda, неизвестный id, длинное название, вторые previews и прокрутка деталей помогают показать нужные состояния. Обычный тест, launcher-ресурсы и backup XML - шаблон проекта. Не надо удалять их, чтобы притвориться, что приложение состоит только из экранов.

Наличие папок SIS1 и SIS2 не лишняя функциональность приложения: они сохраняют предыдущие задания и не входят в source set app. Инструкция «минимум 10» не означает «ровно 10»: 12 допустимо. «Минимум 3» не означает запрет четвёртого экрана, но здесь их ровно три. Preview не считается отдельным экраном пользователя.

Что проверками доказано

assembleDebug, testDebugUnitTest, lintDebug завершились BUILD SUCCESSFUL. Lint: 0 errors, 10 warnings; версии зависимостей, большой вектор, отсутствие monochrome launcher icon. Единственный тест - additionIsCorrect; он не проверяет UI. Состояния и переходы дополнительно проверены через adb на эмуляторе. Это не проверка всех устройств, планшетов, всех масштабов шрифта и восстановления процесса.

> Вывод аудита: функциональные требования SIS3 в основном реализованы. Утверждать «всё выполнено идеально» нельзя из-за коммитов во времени, формальных вопросов дизайна/документации и необходимости твоей самостоятельной защиты. Баллы определяет преподаватель после защиты; автоматический прогноз 100/100 здесь был бы выдумкой.

Практика: изменения, которые ты должен сделать сам

Сначала попробуй выполнить упражнение без подсказки. Затем сравни решение. Проверяй результат в приложении, а не только отсутствие красного подчёркивания. Делай одно упражнение за раз; перед началом сохрани снимок или используй Local History, не откатывай чужие незакоммиченные изменения целиком.

Переименуй заголовок главного экрана

Задание: Измени title_offices в res/values/strings.xml.

Решение для сверки: Открой strings.xml и замени только текст ресурса title_offices. stringResource в ExchangeOfficeListScreen уже указывает на него. Не переименовывай id ресурса без необходимости.

Проверка: После запуска новая надпись в TopAppBar; title_details и title_favorites прежние.

Увеличь расстояние между карточками

Задание: Измени только список на главном экране.

Решение для сверки: В ExchangeOfficeListScreen у LazyColumn замени verticalArrangement = Arrangement.spacedBy(Spacing.medium) на Spacing.large. Не меняй medium в Dimens.kt: иначе изменятся все использующие его места.

Проверка: Промежуток между карточками вырос с 8 до 16 dp; отступы карточки внутри и фильтры не должны поменяться.

Измени все средние отступы одновременно

Задание: Покажи смысл общего Spacing.

Решение для сверки: Теперь измени medium в Dimens.kt с 8.dp на 12.dp как учебный эксперимент. Проследи несколько мест использования. Верни 8.dp после проверки, чтобы сохранить исходную шкалу задания.

Проверка: Меняются промежутки фильтров, курсов и карточек; слово medium - общая настройка, а не одно место.

Поставь название под изображением

Задание: Проверь порядок элементов Column.

Решение для сверки: В Details Image уже стоит перед Text с office.name. Объясни, что задача уже выполнена. Для практики перемести весь блок Text перед Image, посмотри результат, затем верни.

Проверка: Переставляются вертикально именно два блока, а не аргументы одной Image.

Поменяй Buy/Sell местами

Задание: Измени визуальный порядок без ошибки данных.

Решение для сверки: Переставь целиком два вызова RateBlock в нужном Row. Подпись и rate переносятся вместе. Не меняй только labels: иначе подпись будет относиться к неправильному числу.

Проверка: Первым виден Sell USD со значением usdSell.

Покажи полное имя в карточке

Задание: Убери обрезку как эксперимент.

Решение для сверки: В OfficeCard у первого Text убери maxLines = 1 и overflow = Ellipsis. Column.weight(1f) оставь, чтобы сердце имело место. После эксперимента верни обрезку, требуемую SIS3.

Проверка: Длинное имя занимает несколько строк, карточка выше, сердце доступно.

Ограничь заголовок Details двумя строками

Задание: Закрой буквальный вопрос к ellipsis.

Решение для сверки: В Text с office.name добавь maxLines = 2, overflow = TextOverflow.Ellipsis; импорт androidx.compose.ui.text.style.TextOverflow.

Проверка: С id 3 видно не больше двух строк, после переполнения - многоточие.

Добавь новый обменник в Karaganda

Задание: Потренируй модель и listOf.

Решение для сверки: В SampleData.offices добавь ExchangeOffice с уникальным id 13 и city = "Karaganda", всеми семью остальными полями. Строка должна точно совпасть с названием chip.

Проверка: All показывает 13 записей; Karaganda показывает одну; после теста убери запись, если хочешь снова показывать empty state.

Измени primary в светлой теме

Задание: Объясни централизованное оформление.

Решение для сверки: В Color.kt временно замени TealPrimary, например на Color(0xFF1565C0). Не меняй экраны. Смотри светлую тему и верни исходный цвет после упражнения.

Проверка: Активные сердца и primary-кнопка меняются. Векторный рисунок и ночной primary автоматически не обязаны стать синими.

Сделай картинку выше

Задание: Найди размер, не отступ.

Решение для сверки: Измени Sizes.detailsImageHeight в Dimens.kt. height, а не padding, определяет высоту картинки.

Проверка: Изображение выше на Details, контент остаётся прокручиваемым.

Покажи неизвестный id

Задание: Раздели find и null.

Решение для сверки: Временно в OfficeDetailsScreenPreview поставь officeId = 999; для runtime можно временно передать 999 в маршрут карточки, затем вернуть.

Проверка: find возвращает null и отображается Office not found; индекс массива не используется.

Покажи число избранных в заголовке

Задание: Проведи значение в UI.

Решение для сверки: В strings.xml добавь title_favorites_count с текстом Favorites (%1$d). В FavoritesScreen используй stringResource(R.string.title_favorites_count, favorites.size).

Проверка: Удаление последней записи показывает Favorites (0) вместе с EmptyState.

Отсортируй фильтрованный список по покупке

Задание: Не путай фильтрацию с сортировкой.

Решение для сверки: Создай val sortedOffices = offices.sortedByDescending { it.usdBuy } и передай его в items вместо offices.

Проверка: Сначала самое высокое usdBuy. id и корректность перехода не меняются. После опыта верни исходный порядок.

Проверь потерю избранного после restart

Задание: Докажи границу remember.

Решение для сверки: Добавь запись, полностью останови и запусти приложение снова. Затем проверь Favorites. Не подменяй это просто переходом на другой экран.

Проверка: Favorites снова пуст; это текущее поведение, а не поломка callback.

Что особенно легко сломать

Не удаляй padding(innerPadding). Не меняй один officeId на другое имя только в одном из трёх мест маршрута. Не используй индекс вместо id. Не вызывай навигацию сразу в теле UI. Не добавляй второе независимое favoriteIds в FavoritesScreen. Не оставляй color литерал в экране после эксперимента. Не меняй подпись Buy без значения usdBuy. И не говори, что Preview доказывает работоспособность Back: в нём callbacks пустые.

Вопросы на защите: отвечай своими словами

Что такое Scaffold?

Контейнер структуры экрана: верхняя панель и область содержимого. В моём коде topBar содержит TopAppBar, а innerPadding применяется к содержимому.

Почему Column внутри Row?

В карточке слева несколько строк вертикально, справа сердце. Внешний Row ставит эти две части рядом; внутренняя Column ставит название, город и курсы друг под другом.

Чем LazyColumn отличается от Column?

LazyColumn управляет длинным прокручиваемым списком и создаёт нужные элементы по мере необходимости. Column просто размещает своих детей сверху вниз; в Details я отдельно добавил verticalScroll.

Где хранятся данные?

В object SampleData: список ExchangeOffice. Данные локальные и демонстрационные.

Почему data class?

Она описывает одну запись обменника и даёт свойства и стандартные операции над данными. Экран берёт name, city, usdBuy и другие поля одного объекта.

Как id попадает на экран?

Карточка передаёт id в callback. NavGraph строит details/число, маршрут details/{officeId} читает Int-аргумент и передаёт его Details. Там find ищет запись по id.

Почему id, а не индекс?

Порядок и фильтр могут меняться. id идентифицирует обменник, а индекс только позицию в текущем списке.

Как работают избранные?

Общий Set<Int> в NavGraph. toggleFavorite добавляет или удаляет id. Экраны получают этот набор или Boolean, а Favorites фильтрует исходные записи.

Что делает remember?

Удерживает объект в этой позиции composition между пересчётами. Для избранного он не даёт постоянного хранения и не переживает обычное пересоздание Activity.

Что делает mutableStateOf?

Создаёт наблюдаемое значение. Когда я присваиваю новый набор, Compose может пересчитать UI, который читал состояние.

Что такое by?

Чтение и запись переменной делегированы value состояния. getValue/setValue позволяют использовать короткую запись вместо .value.

Где берутся цвета?

Конкретные значения в Color.kt, роли light/dark в Theme.kt. UI читает MaterialTheme.colorScheme.

Что значит onSurface?

Цвет содержимого, которое рисуется на поверхности surface, обычно текста. Это цветовая роль, а не callback.

Что такое Modifier?

Цепочка настроек размера, отступов и рисования элемента. Например fillMaxWidth, height, clip. Порядок влияет на результат.

Почему weight(1f)?

В карточке Column занимает оставшееся место рядом с сердцем. В Details два блока с одинаковым weight делят ширину поровну.

Как работает Back?

Кнопка вызывает callback, а NavGraph делает popBackStack. Возвращаемся к предыдущему посещению, а не всегда жёстко на стартовый экран.

Что такое innerPadding?

Отступы, которые Scaffold вычислил для содержимого с учётом своих панелей и системных краёв. Я применяю их сам.

Зачем contentDescription?

Чтобы экранное чтение объяснило действие иконки или смысл изображения. Для декоративной иконки рядом с поясняющим текстом оно может быть null.

Почему previews не принимают NavController?

Экранам передаются функции событий. В Preview я подставляю пустые лямбды, чтобы смотреть вид без настоящей навигации.

Что происходит с длинным названием?

В OfficeCard maxLines = 1 и Ellipsis показывают многоточие. Данные не обрезаются. В Details название сейчас переносится целиком.

Почему все курсы USD?

Это ограниченный набор демоданных SIS3. Поддержка других валют есть в идее SIS1, но сейчас не реализована.

Что ты сделал сам?

Отвечай только фактами. Если всё генерировал помощник, скажи это и объясни свои реальные решения, изученные части и самостоятельно выполненные правки. Нельзя выдавать предложенную формулировку за уже выполненную работу.

Пробная защита на 6 минут

Минута 1: покажи главный список, длинное имя, Karaganda, Details, добавление в избранное, Favorites и удаление последней записи. Минуты 2-4: без PDF увеличь промежуток между карточками и поменяй порядок Buy/Sell; объясни результат и запусти. Минуты 5-6: объясни маршрут id и разницу remember/mutableStateOf. Затем выбери наугад три строки из проекта и объясни каждую, включая синтаксис.

Критерий готовности: ты можешь не только повторить определение, но и ответить «что сломается, если удалить эту строку» или «где менять поведение». Если пока не можешь, возвращайся к конкретному файлу и снова делай маленькое изменение. На защите autocomplete и официальная документация разрешены; идеальное запоминание синтаксиса не требуется.

SIS2: пять решений Kotlin отдельно от Android

Эти файлы лежат вне app/src/main и не собираются в Android-приложение. В каждом свой class Solution без package. Их предназначение - отдельная отправка в LeetCode. Если собрать все пять в одном source set без разделения, одинаковые классы Solution конфликтуют; ListNode также должен быть предоставлен средой.

Здесь достаточно понять новые Kotlin-конструкции и сам алгоритм. Android, Compose, Scaffold и состояние UI к этим задачам не относятся. Ниже приведён разбор по строкам.

01_TwoSum.kt

Путь: sis2/01_TwoSum.kt

Для nums=[2,7,11,15], target=9: сначала map[2]=0, у 7 дополняющее число 2 найдено, возвращается [0,1].

```
1  // 1. Two Sum — https://leetcode.com/problems/two-sum/
2  // Topics: Array, Hash Map
3  //
4  // Approach: one pass over the array, keeping a map of value -> index for the numbers
5  // already seen. For every number we look up its complement (target - n) in the map:
6  // if it was seen before we found the pair, otherwise we remember the current number.
7  // O(n) time, O(n) memory.
```

Строки 1-7: Комментарии: задача Two Sum, ссылка и подход. Нужны индексы двух чисел, сумма которых target. Храним уже встреченные значения и их индексы в map. O(n) время и O(n) память при среднем быстром поиске в hash map.

```
9  class Solution {
10     fun twoSum(nums: IntArray, target: Int): IntArray {
```

Строки 9-10: class Solution и функция twoSum: принимает IntArray и Int, возвращает IntArray индексов.

```
11         val seen = mutableMapOf<Int, Int>()
```

Строка 11: seen - изменяемый Map<Int,Int>: число -> индекс. val фиксирует ссылку на map, но не запрещает seen[n] = i.

```
12         nums.forEachIndexed { i, n ->
```

Строка 12: forEachIndexed передаёт i (индекс) и n (значение) для каждого элемента массива.

```
13             seen[target - n]?.let { j -> return intArrayOf(j, i) }
```

Строка 13: target-n - недостающее число. seen[...] возвращает его индекс либо null. ?.let выполняет лямбду только при найденном индексе j. return внутри inline let выходит из twoSum, возвращая intArrayOf(j,i). Это не возврат из одной только лямбды.

```
14              seen[n] = i
15          }
```

Строки 14-15: Записываем текущий n после проверки, чтобы не использовать одну позицию дважды. Закрывается forEachIndexed.

```
16          return intArrayOf()
17      }
18  }
```

Строки 16-18: Если пара не найдена, возвращаем пустой IntArray. В контракте задачи пара существует. Закрываются функция и класс.

14_LongestCommonPrefix.kt

Путь: sis2/14_LongestCommonPrefix.kt

reduce - свернуть коллекцию к одному результату. Не filter: результат одна строка, а не подсписок.

```
1  // 14. Longest Common Prefix — https://leetcode.com/problems/longest-common-prefix/
2  // Topics: String
3  //
4  // Approach: the common prefix of the whole array is the common prefix of the first
   string
5  // with the second, then of that result with the third, and so on — exactly a
   `reduce`.
6  // The accumulator starts as strs[0] and is shortened by `commonPrefixWith` for every
   next
7  // string; once it becomes empty it stays empty. O(S) time for S total characters.
```

Строки 1-7: Комментарии: общий префикс массива строк. Последовательно уменьшаем префикс сравнениями. O(S) оценка по суммарным символам.

```
9  class Solution {
10     fun longestCommonPrefix(strs: Array<String>): String {
```

Строки 9-10: Класс и функция с Array<String>, результат String.

```
11         return strs.reduce { prefix, s -> prefix.commonPrefixWith(s) }
```

Строка 11: reduce берёт первый элемент как аккумулятор prefix; для каждой следующей строки s заменяет аккумулятор на prefix.commonPrefixWith(s). Для flower, flow, flight: сначала flow, затем fl.

```
12     }
13 }
```

Строки 12-13: Закрываются функция и класс. Для пустого массива reduce бросит исключение; по контракту LeetCode вход не пустой. Для общего произвольного API эту границу нужно обработать.

20_ValidParentheses.kt

Путь: sis2/20_ValidParentheses.kt

Для «([])»: положили (, положили [, сняли [для], сняли (для). Итог true. Для «([)]» вершина [не подходит к), итог false.

```
1  // 20. Valid Parentheses — https://leetcode.com/problems/valid-parentheses/
2  // Topics: String, Stack
3  //
4  // Approach: a stack of opening brackets — every opening bracket is pushed, every
   closing
5  // bracket must match the bracket on top of the stack, which is popped. A map from
   closing
6  // to opening bracket replaces a chain of if/else, and `all` stops at the first
   mismatch.
7  // The string is valid when everything matched and the stack is empty. O(n) time, O(n)
    memory.
```

Строки 1-7: Комментарии описывают стек открывающих скобок и карту закрывающая -> открывающая. Стек работает по правилу «последняя открытая закрывается первой». Время O(n), память O(n).

```
9  class Solution {
10     fun isValid(s: String): Boolean {
```

Строки 9-10: Класс и функция isValid принимают строку и возвращают Boolean.

```
11         val pairs = mapOf(')' to '(', ']' to '[', '}' to '{')
```

Строка 11: mapOf задаёт соответствия. 'c' - Char, "text" - String. to создаёт пару ключ-значение: ')' to '(' и так далее.

```
12         val stack = ArrayDeque<Char>()
```

Строка 12: ArrayDeque<Char> - двусторонняя очередь; используется как стек через конец.

```
13         val allMatched = s.all { c ->
```

Строка 13: s.all проверяет каждый символ c, останавливается при первом false и возвращает общий Boolean.

```
14             if (c in pairs.values) { stack.addLast(c); true }
```

Строка 14: Если символ есть среди открывающих pairs.values, addLast кладёт его на вершину. После ; стоит true - последнее выражение этой ветки. ; разделяет два действия в одной строке.

```
15             else stack.removeLastOrNull() == pairs[c]
16         }
```

Строки 15-16: Иначе снимаем вершину removeLastOrNull и сравниваем с ожидаемым открывающим символом pairs[c]. Закрывается all. Код опирается на контракт задачи: допустимы только шесть скобок; произвольный символ мог бы дать неверное совпадение null==null.

```
17          return allMatched && stack.isEmpty()
18      }
19  }
```

Строки 17-19: Нужны два условия: все закрывания совпали И stack пуст. && - логическое И с коротким замыканием. Остались открытые - значит строка неверна. Закрываются функция и класс.

21_MergeTwoSortedLists.kt

Путь: sis2/21_MergeTwoSortedLists.kt

Связный список - узлы со ссылкой next, не Kotlin List. Слияние использует существующие узлы; возвращаемая голова связана с новым порядком next.

```
1  // 21. Merge Two Sorted Lists — https://leetcode.com/problems/merge-two-sorted-lists/
2  // Topics: Linked List, Recursion
3  //
4  // Approach: recursive definition — the head of the merged list is the smaller of the
   two
5  // heads, and its tail is the merge of the rest. `when` handles the cases: if one list
    is
6  // empty the answer is the other one, otherwise we keep the smaller node and
   recursively
7  // merge its `next` with the other list (`apply` mutates `next` and returns the node).
8  // O(n + m) time and recursion depth.
```

Строки 1-8: Комментарии: слияние двух отсортированных связных списков через рекурсию. В каждом шаге берём меньшую голову. Время O(n+m), стек вызовов в худшем случае O(n+m).

```
10 /**
11  * Definition for singly-linked list (provided by LeetCode).
12  * class ListNode(var `val`: Int) {
13  *     var next: ListNode? = null
14  * }
15  */
```

Строки 10-15: Многострочный комментарий описывает ListNode, предоставленный LeetCode. Поле `val` взято в обратные кавычки, потому что val - ключевое слово Kotlin. next: ListNode? может указывать на следующий узел либо null. Это описание в комментарии не создаёт настоящий класс.

```
16 class Solution {
17     fun mergeTwoLists(list1: ListNode?, list2: ListNode?): ListNode? {
```

Строки 16-17: Класс Solution. mergeTwoLists принимает две возможные головы либо null и возвращает голову результата либо null.

```
18         return when {
```

Строка 18: when без аргумента последовательно проверяет Boolean-условия; первая подходящая ветка определяет результат.

```
19             list1 == null -> list2
20             list2 == null -> list1
```

Строки 19-20: Если один список закончился, возвращаем оставшуюся голову другого. Новые узлы не создаются.

```
21             list1.`val` <= list2.`val` -> list1.apply { next = mergeTwoLists(next,
    list2) }
```

Строка 21: Если значение list1 меньше/равно list2, выбираем list1. apply выполняет блок с this=list1, присваивает next результату рекурсивного слияния остатка list1 с list2 и возвращает сам list1. Это мутирует исходные узлы.

```
22              else -> list2.apply { next = mergeTwoLists(list1, next) }
```

Строка 22: Иначе выбираем list2 и аналогично рекурсивно соединяем list1 с остатком list2.

```
23          }
24      }
25  }
```

Строки 23-25: Закрываются when, функция и класс. На очень длинных списках рекурсивный вариант может исчерпать стек; это не tail-recursive функция.

242_ValidAnagram.kt

Путь: sis2/242_ValidAnagram.kt

Без Android-зависимостей. Эта строка проверяет частоты, а не сортирует строки, несмотря на общий список тем в комментарии.

```
1  // 242. Valid Anagram — https://leetcode.com/problems/valid-anagram/
2  // Topics: String, Hash Map, Sorting
3  //
4  // Approach: two strings are anagrams when they contain the same characters with the
   same
5  // frequencies. `groupingBy { it }.eachCount()` builds a Map<Char, Int> of character
   counts
6  // for each string in one expression, and Kotlin compares maps structurally with ==.
7  // O(n) time, O(k) memory for k distinct characters.
```

Строки 1-7: Комментарии: анаграммы имеют одинаковые количества каждого символа. Время O(n+m) для двух строк, память по количеству разных символов.

```
9  class Solution {
10     fun isAnagram(s: String, t: String): Boolean {
```

Строки 9-10: Класс и функция с двумя String, результат Boolean.

```
11         return s.groupingBy { it }.eachCount() == t.groupingBy { it }.eachCount()
```

Строка 11: groupingBy { it } группирует символы по самим себе. eachCount превращает группировку в Map<Char,Int>. Две map сравниваются == по ключам и количествам, а не по тому, один ли это объект.

```
12     }
13 }
```

Строки 12-13: Закрываются функция и класс. Например aab и aba имеют {a:2,b:1}, итог true; abb даёт {a:1,b:2}, итог false.

Куда смотреть, если забыл термин

Нужно изменить или понять	Файл / место
Запуск UI	MainActivity.kt
Обменник и его поля	model/ExchangeOffice.kt
Записи, города, длинное имя	data/SampleData.kt
Маршруты, Back, общее избранное	navigation/RateRadarNavGraph.kt
Фильтр, промежуток главного списка	ExchangeOfficeListScreen.kt
Карточка и сердце	OfficeCard.kt
Курс и его подпись	RateBlock.kt
Изображение, адрес, кнопка деталей	OfficeDetailsScreen.kt
Пустое избранное	FavoritesScreen.kt + EmptyState.kt
Конкретный цвет	Color.kt
Соответствие цветовым ролям	Theme.kt
Общие отступы и размеры	Dimens.kt
Надписи интерфейса	res/values/strings.xml
Вектор картинки	res/drawable/exchange_placeholder.xml
Вход для Android	AndroidManifest.xml
Версии библиотек и SDK	app/build.gradle.kts

Официальные источники для сверки API

Основной материал книги получен из твоих файлов и собственного разбора поведения. Указанные ниже страницы проверены для терминов/API; это справочник, а не требование переписать проект на последние версии. Закреплённые версии проекта остаются теми, которые указаны в Gradle.

Kotlin: базовый синтаксис

Compose: состояние

Compose: Scaffold

Compose: модификаторы

Compose: ленивые списки

Android Navigation

Compose: стандартные зоны нажатия и доступность

Учебные требования: SIS1.pdf, 2 страницы; SIS3.pdf, 5 страниц, из твоей Downloads. Фото доски - пример структурного эскиза другого приложения, не требование сделать экран Movies или скопировать его навигацию. Скриншот Outcomes подтверждает правило защиты, которое также есть на странице 4 SIS3.